

FastMCP Production Manual

FastMCP Production Manual — Chapters 2-8 (Paid Content)

FastMCP Production Manual

Chapter 2: Production nginx + TLS

- The Complete nginx Configuration

- Key Directives Explained

- TLS Certificate Automation

- Testing the Setup

- Common Misconfigurations

Chapter 3: Auth in Production

Chapter 4: Docker + Compose

Chapter 5: Security Hardening Checklist

Chapter 6: CI/CD Pipeline

Chapter 7: Monitoring & Alerting

Chapter 8: 30+ Production Errors & Fixes

Appendix A: Corrections & Updates Based on Technical Review

- Correction 1: Version Pinning

- Correction 2: Transport — Streamable HTTP is the Standard

- Correction 3: OAuth 2.1 is Mandatory for Remote MCP Servers

- Correction 4: CVE-2026-48710 — Accurate Version Details

- Correction 5: Prefect Horizon — Zero-Config Deployment

- Alternative

- Correction 6: Cloudflare Workers — Serverless MCP

- Deployment

- Correction 7: Additional OAuth DCR Errors for Chapter 8

- Correction 8: FastMCP 2.x to 3.x Migration Pitfalls

FastMCP Production Manual — Chapters 2-8 (Paid Content)

FastMCP Production Manual

Version: Based on FastMCP 3.4.x (June 2026)**Target Audience:** Developers who have built a local MCP server and need to deploy it to production.**Scope:** Chapters 2-8 (paid content). Chapter 1 (Architecture & Deployment Blueprint) is the free preview published separately.

Chapter 2: Production nginx + TLS

Your FastMCP server is running on uvicorn, answering HTTP requests on localhost. Now you need to put it behind a reverse proxy that handles TLS termination, HTTP/2, rate limiting, and streaming — without breaking MCP's Streamable HTTP transport.

The Complete nginx Configuration

```
# /etc/nginx/sites-available/mcp-server.conf

# Rate limiting zones
limit_req_zone $binary_remote_addr zone=mcp_global:10m rate=30r/s;
limit_req_zone $binary_remote_addr zone=mcp_tools:10m rate=10r/s;
limit_conn_zone $binary_remote_addr zone=mcp_conns:10m;

# Upstream definition
upstream mcp_backend {
    server 127.0.0.1:8000;
    keepalive 32;
}

server {
    listen 80;
    server_name mcp.yourdomain.com;
    return 301 https://$host$request_uri;
}
```

```

server {
    listen 443 ssl http2;
    server_name mcp.yourdomain.com;

    # --- TLS Configuration ---
    ssl_certificate /etc/letsencrypt/live/mcp.yourdomain.com/
fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/mcp.yourdomain.com/
privkey.pem;
    ssl_trusted_certificate /etc/letsencrypt/live/
mcp.yourdomain.com/chain.pem;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-
GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-
SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305;
    ssl_prefer_server_ciphers off;

    ssl_stapling on;
    ssl_stapling_verify on;
    resolver 8.8.8.8 8.8.4.4 valid=300s;

    ssl_session_timeout 1d;
    ssl_session_cache shared:MCP_SSL:10m;
    ssl_session_tickets off;

    # --- Security Headers ---
    add_header Strict-Transport-Security "max-age=63072000;
includeSubDomains; preload" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;

    # --- Request Size Limits ---
    client_max_body_size 16m;
    client_body_buffer_size 128k;

    # --- Timeouts ---
    proxy_connect_timeout 10s;
    proxy_send_timeout 60s;
    proxy_read_timeout 300s;
    send_timeout 60s;

    # --- Connection Limits ---
    limit_conn mcp_conns 20;

    # --- Health Check Endpoint ---
    location = /health {
        proxy_pass http://mcp_backend/health;
        proxy_set_header Host $host;
        access_log off;
    }
}

```

```

# --- OAuth / Auth Endpoints ---
location ~ ^/(\\.well-known/oauth-authorization-server|
authorize|token|register) {
    limit_req zone=mcp_global burst=20 nodelay;
    proxy_pass http://mcp_backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
}

# --- MCP Endpoint (Streamable HTTP) ---
location /mcp {
    limit_req zone=mcp_tools burst=5 nodelay;
    proxy_pass http://mcp_backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Request-ID $request_id;

    # Critical for Streamable HTTP
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_buffering off;
    proxy_cache off;
    chunked_transfer_encoding on;
    proxy_request_buffering off;
    proxy_read_timeout 600s;

    proxy_pass_header mcp-session-id;
    proxy_pass_header mcp-protocol-version;
}

location / {
    return 404;
}
}

```

Key Directives Explained

Streaming Configuration (Critical)

```
proxy_buffering off;  
proxy_request_buffering off;  
chunked_transfer_encoding on;
```

This is where most MCP nginx configs break. FastMCP 3.x uses **Streamable HTTP** — the server sends chunked responses that the client reads incrementally. If nginx buffers these, the client sees nothing until the entire response completes.

proxy_read_timeout

```
proxy_read_timeout 600s;
```

MCP tools can take a long time. A database query tool might run for 30 seconds. A code execution tool might run for minutes. The default 60 seconds will kill these connections. Set high and let the application handle its own timeouts.

CORS Configuration

```
map $http_origin $cors_origin {  
    default "";  
    "https://claude.ai" "https://claude.ai";  
    "https://app.cursor.sh" "https://app.cursor.sh";  
    "~^https://.*\\.yourdomain\\.com$" $http_origin;  
}
```

What breaks silently:

1. Missing mcp-session-id in Expose-Headers → session management fails without error
2. Missing mcp-protocol-version in Allow-Headers → actual request blocked after preflight passes
3. Wildcard * with credentials → browsers silently refuse to send auth headers
4. Missing always → CORS headers disappear on error responses, masking the actual error

TLS Certificate Automation

```
# Install certbot  
sudo apt install -y certbot python3-certbot-nginx
```

```
# Obtain certificate
sudo certbot --nginx -d mcp.yourdomain.com --non-interactive --
agree-tos --email ops@yourdomain.com

# Verify auto-renewal
sudo systemctl status certbot.timer

# Cron renewal with nginx reload
echo "0 3 * * * root certbot renew --quiet --deploy-hook
'systemctl reload nginx'" > /etc/cron.d/certbot-renewal
```

Testing the Setup

```
# Test streaming works (should output immediately, not buffer)
curl -N -X POST https://mcp.yourdomain.com/mcp \
-H "Content-Type: application/json" \
-H "Authorization: Bearer YOUR_TOKEN" \
-H "mcp-protocol-version: 2025-11-05" \
-d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}' --max-
time 5

# Test TLS
openssl s_client -connect mcp.yourdomain.com:443 -servername
mcp.yourdomain.com < /dev/null 2>/dev/null | grep -E "(Protocol|
Cipher)"

# Test rate limiting
for i in $(seq 1 20); do
    echo -n "Request $i: "
    curl -s -o /dev/null -w "%{http_code}" -X POST https://
mcp.yourdomain.com/mcp -H "Content-Type: application/json" -d
'{"jsonrpc":"2.0","id":1,"method":"ping"}'
    echo
done
```

Common Misconfigurations

Symptom	Cause	Fix
Client sees empty response after long wait	proxy_buffering on (default)	Add proxy_buffering off
504 Gateway Timeout on tool calls	proxy_read_timeout 60s (default)	Increase to 300-600s
		Add always

Symptom	Cause	Fix
CORS error only on errors (401, 500)	Missing always on add_header	
Session lost between requests	mcp-session-id not exposed	Add to Access-Control-Expose-Headers
Random 502 Bad Gateway	Upstream keepalive misconfigured	Ensure proxy_http_version 1.1 + clear Connection

Chapter 3: Auth in Production

Authentication in MCP isn't optional — the protocol specification mandates OAuth 2.1 for remote servers. This chapter covers every auth pattern you'll encounter: from simple bearer tokens for internal services to full OAuth 2.1 with Dynamic Client Registration for public MCP servers.

Understanding MCP Auth Requirements

The MCP specification (2025-11-05 revision) requires:

1. **OAuth 2.1** as the authorization framework
2. **Dynamic Client Registration (DCR)** so MCP clients can self-register
3. **PKCE** (Proof Key for Code Exchange) on all authorization code flows
4. **Protected Resource Metadata** (RFC 9728) for server discovery
5. **Resource Indicators** (RFC 8707) for multi-tenant deployments

Claude, Cursor, and other major MCP clients expect these. If your server doesn't implement them, clients can't connect.

Pattern 1: Bearer Token Auth (Internal Services)

For internal microservices or trusted environments where you control both client and server, simple bearer token validation is the fastest path.

Implementation with FastMCP 3.x

```
# auth/bearer.py
from fastmcp import FastMCP
from fastmcp.server.auth import import BearerAuthProvider
from starlette.requests import Request
from starlette.exceptions import HTTPException
import hashlib
import hmac
import os

# Token store – in production, use Redis or a database
VALID_TOKENS = {
    # Store hashed tokens, not plaintext
    hashlib.sha256(b"your-service-token-here").hexdigest(): {
        "service": "data-pipeline",
        "scopes": ["tools:read", "tools:execute"],
    }
}

class InternalBearerAuth(BearerAuthProvider):
    """Bearer token validation for internal services."""

    async def authenticate(self, request: Request) -> dict:
        auth_header = request.headers.get("Authorization", "")
        if not auth_header.startswith("Bearer "):
            raise HTTPException(status_code=401, detail="Missing
bearer token")

        token = auth_header[7:]
        token_hash = hashlib.sha256(token.encode()).hexdigest()

        if token_hash not in VALID_TOKENS:
            raise HTTPException(status_code=401, detail="Invalid
token")

        return VALID_TOKENS[token_hash]

mcp = FastMCP(
    "Internal MCP Server",
    auth=InternalBearerAuth(),
)

@mcp.tool()
async def query_database(sql: str) -> str:
    """Execute a read-only SQL query."""
    # auth context is available via request state
```

```
# ...
return "results"
```

Token Generation and Rotation

```
# scripts/generate_token.py
import secrets
import hashlib
import json
from datetime import datetime, timezone

def generate_service_token(service_name: str, scopes: list[str]) -
> dict:
    """Generate a new service token and its hash for storage."""
    token = secrets.token_urlsafe(32)
    token_hash = hashlib.sha256(token.encode()).hexdigest()

    return {
        "token": token, # Give this to the service – never store
it
        "hash": token_hash, # Store this in your token database
        "service": service_name,
        "scopes": scopes,
        "created_at": datetime.now(timezone.utc).isoformat(),
    }

if __name__ == "__main__":
    result = generate_service_token("data-pipeline",
["tools:read", "tools:execute"])
    print(f"Token (distribute securely): {result['token']}")
    print(f"Hash (store in database): {result['hash']}")
    print(f"Full record:\n{json.dumps(result, indent=2)}")
```

Pattern 2: JWT Validation (Existing Identity Infrastructure)

When you already have an identity provider (Auth0, Okta, Azure AD) issuing JWTs, validate them at the MCP server without implementing the full OAuth dance yourself.

Full JWT Middleware

```
# auth/jwt_auth.py
import httpx
import jwt
from jwt import PyJWKClient
from fastmcp import FastMCP
from fastmcp.server.auth import BearerAuthProvider
from starlette.requests import Request
```

```

from starlette.exceptions import HTTPException
from functools import lru_cache
from datetime import datetime, timezone
import logging

logger = logging.getLogger(__name__)

class JWTAuthProvider(BearerAuthProvider):
    """
    JWT validation against an external OIDC provider.

    Supports:
    - JWKS key rotation (automatic refresh)
    - Issuer and audience validation
    - Scope-based access control
    - Token expiry checking
    """

    def __init__(
        self,
        issuer: str,
        audience: str,
        jwks_uri: str | None = None,
        required_scopes: list[str] | None = None,
    ):
        self.issuer = issuer
        self.audience = audience
        self.required_scopes = required_scopes or []

        # Auto-discover JWKS URI from OIDC metadata if not
        provided

        if jwks_uri is None:
            jwks_uri = f"{issuer.rstrip('/')}/.well-known/openid-
configuration"

        self._jwks_client = PyJWKClient(
            jwks_uri,
            cache_jwk_set=True,
            lifespan=3600, # Cache JWKS for 1 hour
        )

    async def authenticate(self, request: Request) -> dict:
        auth_header = request.headers.get("Authorization", "")
        if not auth_header.startswith("Bearer "):
            raise HTTPException(
                status_code=401,
                detail="Missing or malformed Authorization
header",

                headers={"WWW-Authenticate": "Bearer"},
            )

```

```

token = auth_header[7:]

try:
    # Get the signing key from JWKS
    signing_key =
self._jwks_client.get_signing_key_from_jwt(token)

    # Decode and validate
    payload = jwt.decode(
        token,
        signing_key.key,
        algorithms=["RS256", "ES256"],
        issuer=self.issuer,
        audience=self.audience,
        options={
            "verify_exp": True,
            "verify_iat": True,
            "verify_iss": True,
            "verify_aud": True,
            "require": ["exp", "iss", "sub"],
        },
    )

except jwt.ExpiredSignatureError:
    raise HTTPException(
        status_code=401,
        detail="Token expired",
        headers={"WWW-Authenticate": 'Bearer
error="invalid_token"'},
    )
except jwt.InvalidIssuerError:
    raise HTTPException(status_code=401, detail="Invalid
token issuer")
except jwt.InvalidAudienceError:
    raise HTTPException(status_code=401, detail="Invalid
token audience")
except jwt.PyJWTError as e:
    logger.warning(f"JWT validation failed: {e}")
    raise HTTPException(status_code=401, detail="Invalid
token")

# Check required scopes
token_scopes = set(payload.get("scope", "").split())
required = set(self.required_scopes)
if required and not required.issubset(token_scopes):
    missing = required - token_scopes
    raise HTTPException(
        status_code=403,
        detail=f"Missing required scopes: {'',
'.join(missing)}",
    )

```

```

        return {
            "sub": payload["sub"],
            "email": payload.get("email"),
            "scopes": list(token_scopes),
            "teams": payload.get("teams", []),
        }

# Usage
mcp = FastMCP(
    "Production MCP Server",
    auth=JWTAuthProvider(
        issuer="https://your-tenant.auth0.com/",
        audience="https://mcp.yourdomain.com",
        required_scopes=["mcp:tools"],
    ),
)

```

Token Refresh Pattern for MCP Clients

MCP clients need to handle token refresh transparently. Here's a client-side pattern:

```

# client/token_manager.py
import httpx
import time
from dataclasses import dataclass
from threading import Lock

@dataclass
class TokenPair:
    access_token: str
    refresh_token: str
    expires_at: float # Unix timestamp

class TokenManager:
    """
    Manages OAuth tokens for an MCP client.
    Handles refresh before expiry with a safety buffer.
    """

    def __init__(
        self,
        token_endpoint: str,
        client_id: str,
        client_secret: str | None = None,
        refresh_buffer_seconds: int = 60,
    ):
        self.token_endpoint = token_endpoint

```

```

self.client_id = client_id
self.client_secret = client_secret
self.refresh_buffer = refresh_buffer_seconds
self._token: TokenPair | None = None
self._lock = Lock()

@property
def access_token(self) -> str:
    """Get a valid access token, refreshing if needed."""
    with self._lock:
        if self._token is None:
            raise RuntimeError("No token available. Call
authenticate() first.")

        if time.time() >= (self._token.expires_at -
self.refresh_buffer):
            self._refresh()

        return self._token.access_token

def _refresh(self):
    """Exchange refresh token for a new access token."""
    data = {
        "grant_type": "refresh_token",
        "refresh_token": self._token.refresh_token,
        "client_id": self.client_id,
    }
    if self.client_secret:
        data["client_secret"] = self.client_secret

    response = httpx.post(self.token_endpoint, data=data)
    response.raise_for_status()

    token_data = response.json()
    self._token = TokenPair(
        access_token=token_data["access_token"],
        refresh_token=token_data.get("refresh_token",
self._token.refresh_token),
        expires_at=time.time() + token_data["expires_in"],
    )

```

Pattern 3: Full OAuth 2.1 with Dynamic Client Registration

This is what you need for public MCP servers that Claude, Cursor, and other clients connect to. FastMCP 3.x provides built-in support via RemoteAuthProvider.

Using FastMCP's Built-in OAuth

```
# server.py
from fastmcp import FastMCP
from fastmcp.server.auth import RemoteAuthProvider

mcp = FastMCP(
    "Public MCP Server",
    auth=RemoteAuthProvider(
        # Your OAuth provider's issuer URL
        issuer_url="https://auth.yourdomain.com",
        # Where clients should register themselves (DCR endpoint)
        client_registration_url="https://auth.yourdomain.com/
register",
        # Required scopes for MCP access
        scopes=["mcp:tools", "mcp:resources"],
    ),
)
```

Manual OAuth 2.1 Implementation (When You Need Control)

For full control over the OAuth flow — custom consent screens, team-based access, audit logging:

```
# auth/oauth.py
import secrets
import hashlib
import base64
import time
from urllib.parse import urlencode
from fastmcp import FastMCP
from starlette.applications import Starlette
from starlette.requests import Request
from starlette.responses import JSONResponse, RedirectResponse
from starlette.routing import Route
import redis.asyncio as redis

# Redis for token/session storage
redis_client = redis.from_url("redis://localhost:6379/0")

# --- OAuth Metadata Discovery (RFC 8414) ---
async def oauth_metadata(request: Request) -> JSONResponse:
    """
    /.well-known/oauth-authorization-server
    MCP clients hit this first to discover auth endpoints.
    """
    base_url = str(request.base_url).rstrip("/")
    return JSONResponse({
        "issuer": base_url,
```

```

        "authorization_endpoint": f"{base_url}/authorize",
        "token_endpoint": f"{base_url}/token",
        "registration_endpoint": f"{base_url}/register",
        "response_types_supported": ["code"],
        "grant_types_supported": ["authorization_code",
"refresh_token"],
        "token_endpoint_auth_methods_supported": ["none",
"client_secret_post"],
        "code_challenge_methods_supported": ["S256"],
        "scopes_supported": ["mcp:tools", "mcp:resources",
"mcp:prompts"],
    })

```

```

# --- Dynamic Client Registration (RFC 7591) ---
async def register_client(request: Request) -> JSONResponse:
    """
    POST /register
    MCP clients register themselves automatically.
    """
    body = await request.json()

    client_id = f"mcp_{secrets.token_urlsafe(16)}"
    client_secret = secrets.token_urlsafe(32)

    # Store client registration
    await redis_client.hset(
        f"oauth:client:{client_id}",
        mapping={
            "client_secret": client_secret,
            "client_name": body.get("client_name", "Unknown MCP
Client"),
            "redirect_uris": ", ".join(body.get("redirect_uris",
[])),
            "grant_types": ", ".join(body.get("grant_types",
["authorization_code"])),
            "registered_at": str(int(time.time())),
        },
    )
    await redis_client.expire(f"oauth:client:{client_id}", 86400
* 30) # 30 days

    return JSONResponse(
        {
            "client_id": client_id,
            "client_secret": client_secret,
            "client_name": body.get("client_name"),
            "redirect_uris": body.get("redirect_uris", []),
            "grant_types": body.get("grant_types",
["authorization_code"]),
        },
        status_code=201,
    )

```

```

    )

    # --- Authorization Endpoint ---
    async def authorize(request: Request) -> RedirectResponse:
        """
        GET /authorize
        Handles PKCE authorization code flow.
        """

        params = request.query_params
        client_id = params.get("client_id")
        redirect_uri = params.get("redirect_uri")
        code_challenge = params.get("code_challenge")
        code_challenge_method = params.get("code_challenge_method",
"S256")

        state = params.get("state")
        scope = params.get("scope", "")

        # Validate client exists
        client_data = await redis_client.hgetall(f"oauth:client:
{client_id}")
        if not client_data:
            return JSONResponse({"error": "invalid_client"},
status_code=400)

        # Validate redirect URI
        allowed_uris = client_data.get(b"redirect_uris",
b"").decode().split(",")
        if redirect_uri not in allowed_uris:
            return JSONResponse({"error": "invalid_redirect_uri"},
status_code=400)

        # PKCE is required
        if not code_challenge or code_challenge_method != "S256":
            return JSONResponse({"error": "invalid_request",
"error_description": "PKCE with S256 is required"},
status_code=400)

        # Generate authorization code
        auth_code = secrets.token_urlsafe(32)

        # Store auth code with PKCE challenge
        await redis_client.hset(
            f"oauth:code:{auth_code}",
            mapping={
                "client_id": client_id,
                "redirect_uri": redirect_uri,
                "code_challenge": code_challenge,
                "scope": scope,
                "created_at": str(int(time.time())),
            },
        )
    )

```

```

        await redis_client.expire(f"oauth:code:{auth_code}", 600) #
10 minute expiry

        # Redirect back with code
        redirect_params = urlencode({"code": auth_code, "state":
state})

        return HttpResponseRedirect(f"{redirect_uri}?{redirect_params}")

# --- Token Endpoint ---
async def token_endpoint(request: Request) -> JSONResponse:
    """
    POST /token
    Exchange authorization code for access/refresh tokens.
    Validates PKCE code_verifier.
    """
    form = await request.form()
    grant_type = form.get("grant_type")

    if grant_type == "authorization_code":
        return await _handle_auth_code(form)
    elif grant_type == "refresh_token":
        return await _handle_refresh(form)
    else:
        return JSONResponse({"error": "unsupported_grant_type"},
status_code=400)

async def _handle_auth_code(form) -> JSONResponse:
    code = form.get("code")
    code_verifier = form.get("code_verifier")
    client_id = form.get("client_id")

    # Retrieve stored auth code
    code_data = await redis_client.hgetall(f"oauth:code:{code}")
    if not code_data:
        return JSONResponse({"error": "invalid_grant"},
status_code=400)

    # Verify client
    if code_data.get(b"client_id", b "").decode() != client_id:
        return JSONResponse({"error": "invalid_client"},
status_code=400)

    # Verify PKCE
    code_challenge = code_data.get(b"code_challenge",
b "").decode()
    expected_challenge = base64.urlsafe_b64encode(
        hashlib.sha256(code_verifier.encode()).digest()
    ).rstrip(b"=").decode()

    if not secrets.compare_digest(code_challenge,

```

```

expected_challenge):
    return JsonResponse({"error": "invalid_grant",
        "error_description": "PKCE verification failed"}, status_code=400)

    # Delete used auth code (one-time use)
    await redis_client.delete(f"oauth:code:{code}")

    # Generate tokens
    access_token = secrets.token_urlsafe(32)
    refresh_token = secrets.token_urlsafe(32)
    expires_in = 3600 # 1 hour

    scope = code_data.get(b"scope", b"").decode()

    # Store tokens
    await redis_client.hset(
        f"oauth:token:{access_token}",
        mapping={
            "client_id": client_id,
            "scope": scope,
            "type": "access",
            "created_at": str(int(time.time())),
        },
    )
    expires_in)
    await redis_client.expire(f"oauth:token:{access_token}",
        86400 * 7) # 7 days

    await redis_client.hset(
        f"oauth:refresh:{refresh_token}",
        mapping={
            "client_id": client_id,
            "scope": scope,
            "access_token": access_token,
        },
    )
    await redis_client.expire(f"oauth:refresh:{refresh_token}",
        86400 * 7) # 7 days

    return JsonResponse({
        "access_token": access_token,
        "token_type": "Bearer",
        "expires_in": expires_in,
        "refresh_token": refresh_token,
        "scope": scope,
    })

async def _handle_refresh(form) -> JsonResponse:
    refresh_token = form.get("refresh_token")
    client_id = form.get("client_id")

    refresh_data = await redis_client.hgetall(f"oauth:refresh:

```

```

{refresh_token}")
    if not refresh_data:
        return JSONResponse({"error": "invalid_grant"},
status_code=400)

    if refresh_data.get(b"client_id", b"").decode() != client_id:
        return JSONResponse({"error": "invalid_client"},
status_code=400)

    # Revoke old access token
    old_access = refresh_data.get(b"access_token", b"").decode()
    await redis_client.delete(f"oauth:token:{old_access}")

    # Generate new tokens (token rotation)
    new_access = secrets.token_urlsafe(32)
    new_refresh = secrets.token_urlsafe(32)
    scope = refresh_data.get(b"scope", b"").decode()
    expires_in = 3600

    await redis_client.hset(
        f"oauth:token:{new_access}",
        mapping={"client_id": client_id, "scope": scope, "type":
"access", "created_at": str(int(time.time()))},
    )
    await redis_client.expire(f"oauth:token:{new_access}",
expires_in)

    # Rotate refresh token
    await redis_client.delete(f"oauth:refresh:{refresh_token}")
    await redis_client.hset(
        f"oauth:refresh:{new_refresh}",
        mapping={"client_id": client_id, "scope": scope,
"access_token": new_access},
    )
    await redis_client.expire(f"oauth:refresh:{new_refresh}",
86400 * 7)

    return JSONResponse({
        "access_token": new_access,
        "token_type": "Bearer",
        "expires_in": expires_in,
        "refresh_token": new_refresh,
        "scope": scope,
    })

```

Token Persistence: Redis vs File-Based

Factor	Redis	File-Based (SQLite)
Performance	Sub-millisecond lookups	1-5ms (still fine)
Scaling		Single-process only

Factor	Redis	File-Based (SQLite)
	Multiple workers/ containers share state	
Expiry	Built-in TTL with EXPIRE	Manual cleanup needed
Reliability	Requires Redis process running	Zero dependencies
Best for	Multi-worker uvicorn, Docker, K8s	Single-process dev, small deploys

Rule of thumb: If you run uvicorn with `--workers > 1` or use Docker with replicas, use Redis. Otherwise, SQLite is simpler and sufficient.

Session Management for Multi-Client MCP Servers

MCP sessions are identified by the `mcp-session-id` header. Each session maintains its own state (subscriptions, context, capabilities). In production with multiple workers:

```
# session/redis_store.py
import redis.asyncio as redis
import json
from datetime import datetime, timezone

class RedisSessionStore:
    """
    Distributed session store for multi-worker MCP deployments.
    Each uvicorn worker can access any session.
    """

    def __init__(self, redis_url: str = "redis://localhost:
6379/1"):
        self.redis = redis.from_url(redis_url)
        self.session_ttl = 3600 * 4 # 4 hours

    async def create_session(self, session_id: str, client_id:
str, metadata: dict) -> None:
        session_data = {
            "client_id": client_id,
            "created_at": datetime.now(timezone.utc).isoformat(),
            "last_active": datetime.now(timezone.utc).isoformat(),
            "metadata": metadata,
        }
        await self.redis.set(
            f"mcp:session:{session_id}",
            json.dumps(session_data),
            ex=self.session_ttl,
        )
```

```

    async def get_session(self, session_id: str) -> dict | None:
        data = await self.redis.get(f"mcp:session:{session_id}")
        if data is None:
            return None

        session = json.loads(data)
        # Touch: update last_active and reset TTL
        session["last_active"] =
datetime.now(timezone.utc).isoformat()
        await self.redis.set(
            f"mcp:session:{session_id}",
            json.dumps(session),
            ex=self.session_ttl,
        )
        return session

    async def delete_session(self, session_id: str) -> None:
        await self.redis.delete(f"mcp:session:{session_id}")

    async def list_active_sessions(self, client_id: str) ->
list[str]:
        """List all active sessions for a client (for admin/
debugging)."""
        sessions = []
        async for key in
self.redis.scan_iter(match="mcp:session:*"):
            data = await self.redis.get(key)
            if data:
                session = json.loads(data)
                if session.get("client_id") == client_id:
                    sessions.append(key.decode().split(":")[-1])
        return sessions

```

Multi-Team SSO: Routing Requests to the Right Team's Tools

In multi-tenant deployments, different teams should only access their own tools:

```

# auth/team_router.py
from fastmcp import FastMCP
from fastmcp.server.auth import BearerAuthProvider

class TeamScopedAuth(BearerAuthProvider):
    """
    Validates JWT and extracts team membership.
    Tools check team membership before executing.
    """

```

```

    async def authenticate(self, request) -> dict:
        # ... JWT validation (same as Pattern 2) ...
        payload = await self._validate_jwt(request)

        return {
            "sub": payload["sub"],
            "teams": payload.get("teams", []),
            "org_id": payload.get("org_id"),
        }

mcp = FastMCP("Multi-Team MCP", auth=TeamScopedAuth())

def require_team(team: str):
    """Decorator to restrict a tool to a specific team."""
    def decorator(func):
        original = func
        async def wrapper(*args, **kwargs):
            from fastmcp.server.dependencies import import_get_auth_context

            auth = get_auth_context()
            if team not in auth.get("teams", []):
                raise PermissionError(f"Tool requires membership in team: {team}")

            return await original(*args, **kwargs)
        wrapper.__name__ = original.__name__
        wrapper.__doc__ = original.__doc__
        return wrapper
    return decorator

@mcp.tool()
@require_team("engineering")
async def deploy_service(service: str, version: str) -> str:
    """Deploy a service to staging."""
    return f"Deployed {service}:{version} to staging"

@mcp.tool()
@require_team("data-science")
async def run_experiment(experiment_id: str) -> str:
    """Trigger an ML experiment run."""
    return f"Started experiment {experiment_id}"

```

CVE-2026-48710 Mitigation

Vulnerability: Starlette's `request.url.path` can be poisoned via a malformed `Host` header, bypassing path-based security checks.

Affected versions: Starlette < 1.0.1, FastAPI < 0.116.0

Detection:

```
# Check your Starlette version
pip show starlette | grep Version

# Test for vulnerability
curl -H "Host: evil.com/health#" https://your-server.com/admin -v
# If you get 200 instead of 401, you're vulnerable
```

Fix:

```
# Option 1: Upgrade (recommended)
pip install "starlette>=1.0.1"
# or
pip install "fastapi>=0.116.0"

# Option 2: Pin in requirements.txt
echo "starlette>=1.0.1" >> requirements.txt
```

Workaround (if you can't upgrade immediately):

```
# middleware/host_validation.py
import re
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import Response

# RFC 3986 compliant host pattern
HOST_PATTERN = re.compile(r"^[a-zA-Z0-9._~: @!<equation>&'()*+,-;=]
+</equation>")

class HostValidationMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        host = request.headers.get("host", "")
        # Strip port if present
        host_name = host.split(":")[0] if ":" in host else host

        if not HOST_PATTERN.match(host_name):
            return Response("Invalid Host header",
status_code=400)

        # Also check for path traversal characters in host
        if any(c in host for c in "?#"):
            return Response("Invalid Host header",
status_code=400)

        return await call_next(request)
```

Critical: Always use `request.scope["path"]` for security decisions, never `request.url.path`. The scope path comes from the ASGI server's URL parsing, which isn't affected by the Host header.

Auth Failure Debugging

Error Message	Actual Cause	Fix
401: Bearer error="invalid_token"	Token expired or JWKS key rotated	Check token expiry; verify JWKS endpoint returns current keys
403: Insufficient scope	Token is valid but missing required scopes	Request token with correct scopes; check IdP scope configuration
400: invalid_client during DCR	Registration endpoint rejecting client metadata	Check <code>redirect_uris</code> format; ensure <code>grant_types</code> includes <code>authorization_code</code>
400: invalid_grant during token exchange	Auth code expired (>10min) or already used	Codes are single-use; check for retry logic replaying codes
CORS error on /token	Token endpoint missing CORS headers	Ensure nginx adds CORS headers to auth routes (see Chapter 2)
401 on every request despite valid token	Middleware checking wrong path (CVE-2026-48710)	Upgrade Starlette; use <code>scope["path"]</code> not <code>request.url.path</code>
TypeError: 'NoneType' object is not subscriptable in auth	<code>request.headers.get()</code> returning None, then indexing	Use <code>get("Authorization", "")</code> with default empty string
Token refresh returns invalid_grant	Refresh token was already rotated (replay attack or race condition)	Implement refresh token families; allow brief grace period for concurrent refreshes

Chapter 4: Docker + Compose

A FastMCP server in Docker needs more thought than a typical Python web app. MCP sessions are stateful, tool execution can be long-running, and health checks need to verify more than HTTP connectivity.

Multi-Stage Dockerfile

```
# Dockerfile
# Stage 1: Build dependencies
FROM python:3.12-slim AS builder

WORKDIR /build

# Install build dependencies
RUN apt-get update && \
    apt-get install -y --no-install-recommends gcc libffi-dev && \
    rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r
requirements.txt

# Stage 2: Production image
FROM python:3.12-slim AS production

# Security: non-root user
RUN groupadd -r mcp && useradd -r -g mcp -d /app -s /sbin/nologin
mcp

WORKDIR /app

# Copy installed packages from builder
COPY --from=builder /install /usr/local

# Copy application code
COPY --chown=mcp:mcp . .

# Remove unnecessary files
RUN find . -type d -name __pycache__ -exec rm -rf {} + 2>/dev/
null || true && \
    find . -type f -name "*.pyc" -delete

# Switch to non-root user
USER mcp

# Health check
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --
retries=3 \
    CMD python -c "import httpx; r = httpx.get('http://localhost:
8000/health'); assert r.status_code == 200 and
r.json().get('status') == 'healthy'"

# Expose port
EXPOSE 8000
```

```

# Run with uvicorn
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port",
"8000", "--workers", "4", "--loop", "uvloop", "--http",
"httptools"]

```

Health Checks That Actually Verify MCP Readiness

A health check that just returns 200 tells you nothing. Your MCP server might be up but unable to list tools, connect to databases, or accept sessions.

```

# health.py
import time
from fastmcp import FastMCP
from starlette.requests import Request
from starlette.responses import JSONResponse
from starlette.routing import Route

async def health_check(request: Request) -> JSONResponse:
    """
    Comprehensive health check for MCP server.
    Returns 200 only if the server can actually serve MCP
requests.
    """
    checks = {}
    healthy = True

    # Check 1: MCP server can list tools
    try:
        tools = await mcp._tool_manager.list_tools()
        checks["tools"] = {"status": "ok", "count": len(tools)}
    except Exception as e:
        checks["tools"] = {"status": "error", "error": str(e)}
        healthy = False

    # Check 2: Redis connectivity (for sessions)
    try:
        from session.redis_store import redis_client
        await redis_client.ping()
        checks["redis"] = {"status": "ok"}
    except Exception as e:
        checks["redis"] = {"status": "error", "error": str(e)}
        healthy = False

    # Check 3: External dependencies (database, APIs)
    try:
        from db import get_connection
        async with get_connection() as conn:
            await conn.execute("SELECT 1")
        checks["database"] = {"status": "ok"}
    
```

```

except Exception as e:
    checks["database"] = {"status": "error", "error": str(e)}
    healthy = False

# Check 4: Memory usage (prevent OOM kills)
import resource
mem_mb = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss /
1024

checks["memory_mb"] = mem_mb
if mem_mb > 512: # Alert if over 512MB
    checks["memory"] = {"status": "warning", "usage_mb":
mem_mb}

status_code = 200 if healthy else 503
return JsonResponse(
    {
        "status": "healthy" if healthy else "unhealthy",
        "timestamp": time.time(),
        "checks": checks,
    },
    status_code=status_code,
)

# Liveness probe (simpler – just "is the process alive?")
async def liveness(request: Request) -> JsonResponse:
    return JsonResponse({"status": "alive"})

```

Non-Root User Setup

The Dockerfile already creates a mcp user, but you need to handle permissions for:

```

# If your app writes to a data directory:
RUN mkdir -p /app/data && chown mcp:mcp /app/data

# If you need to read secrets from a mounted volume:
# The volume must be readable by UID 1000 (or whatever the mcp
user's UID is)
# In docker-compose.yml, use: user: "1000:1000"

```

Common pitfall: If your tool writes temp files, ensure /tmp is writable:

```

# Create a writable tmp directory owned by the mcp user
RUN mkdir -p /tmp/mcp && chown mcp:mcp /tmp/mcp
ENV TMPDIR=/tmp/mcp

```

Secrets Management

Method	Use Case	Security Level
Environment variables	Non-sensitive config (ports, hosts)	Low — visible in <code>docker inspect</code>
<code>.env</code> file + <code>env_file</code> :	Development only	Low — committed by accident
Docker Secrets	Swarm mode production	High — mounted as files, never in <code>inspect</code>
Build args	Build-time only (e.g., private PyPI URL)	Medium — stored in image layers
External vault (HashiCorp, AWS SM)	Enterprise production	Highest — runtime fetch with rotation

Docker Secrets Example

```
# docker-compose.yml (Swarm mode)
services:
  mcp:
    image: your-mcp-server:latest
    secrets:
      - db_password
      - redis_url
      - oauth_client_secret
    environment:
      - DB_PASSWORD_FILE=/run/secrets/db_password
      - REDIS_URL_FILE=/run/secrets/redis_url

secrets:
  db_password:
    external: true
  redis_url:
    external: true
  oauth_client_secret:
    external: true
```

Reading secrets in your application:

```
# config.py
import os
from pathlib import Path

def get_secret(name: str) -> str:
    """
    Read a secret from Docker secrets file or environment
    variable.
```

```

Supports both /run/secrets/NAME and NAME_FILE=/path patterns.
"""
# Check Docker secrets path first
secret_path = Path(f"/run/secrets/{name}")
if secret_path.exists():
    return secret_path.read_text().strip()

# Check _FILE environment variable
file_env = os.environ.get(f"{name.upper()}_FILE")
if file_env and Path(file_env).exists():
    return Path(file_env).read_text().strip()

# Fall back to direct environment variable
value = os.environ.get(name.upper())
if value:
    return value

raise ValueError(f"Secret
'{name}' not found in secrets, _FILE env, or direct env")

# Usage
DB_PASSWORD = get_secret("db_password")
REDIS_URL = get_secret("redis_url")

```

Full docker-compose.yml

```

# docker-compose.yml
version: "3.8"

services:
  # --- MCP Application Server ---
  mcp:
    build:
      context: .
      dockerfile: Dockerfile
    restart: unless-stopped
    environment:
      - REDIS_URL=redis://redis:6379/0
      - LOG_LEVEL=info
      - WORKERS=4
      - MCP_SERVER_NAME=production-mcp
    env_file:
      - .env.production # Non-secret config only
    depends_on:
      redis:
        condition: service_healthy
    networks:
      - mcp-internal
    deploy:
      resources:

```

```

    limits:
      memory: 1G
      cpus: "2.0"
    reservations:
      memory: 256M
      cpus: "0.5"
  logging:
    driver: json-file
    options:
      max-size: "50m"
      max-file: "5"
# Don't expose port directly – nginx handles external traffic
  expose:
    - "8000"

# --- nginx Reverse Proxy ---
nginx:
  image: nginx:1.27-alpine
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx/conf.d:/etc/nginx/conf.d:ro
    - ./nginx/ssl:/etc/nginx/ssl:ro
    - certbot-webroot:/var/www/certbot:ro
    - certbot-certs:/etc/letsencrypt:ro
  depends_on:
    - mcp
  networks:
    - mcp-internal
  healthcheck:
    test: ["CMD", "nginx", "-t"]
    interval: 30s
    timeout: 5s
    retries: 3

# --- Redis (Session Store) ---
redis:
  image: redis:7-alpine
  restart: unless-stopped
  command: >
    redis-server
    --maxmemory 256mb
    --maxmemory-policy allkeys-lru
    --save 60 1000
    --appendonly yes
  volumes:
    - redis-data:/data
  networks:
    - mcp-internal
  healthcheck:

```

```

        test: ["CMD", "redis-cli", "ping"]
        interval: 10s
        timeout: 3s
        retries: 5
    deploy:
        resources:
            limits:
                memory: 300M

# --- Certbot (TLS Certificates) ---
certbot:
    image: certbot/certbot:latest
    volumes:
        - certbot-webroot:/var/www/certbot
        - certbot-certs:/etc/letsencrypt
    # Run renewal check every 12 hours
    entrypoint: "/bin/sh -c 'trap exit TERM; while :; do certbot
renew --webroot --webroot-path=/var/www/certbot; sleep 12h & wait
${!}; done'"

volumes:
    redis-data:
    certbot-webroot:
    certbot-certs:

networks:
    mcp-internal:
        driver: bridge

```

Docker Networking

Bridge (Default — Single Host)

```

networks:
    mcp-internal:
        driver: bridge

```

- All containers on the same host communicate via container names as DNS
- mcp can reach redis at redis:6379
- External access only through published ports on nginx

Overlay (Swarm — Multi-Host)

```

networks:
    mcp-internal:
        driver: overlay

```

```
attachable: true
encrypted: true # Encrypt traffic between nodes
```

Use overlay when:

- You need multiple replicas of the MCP server across hosts
- You want Swarm's built-in load balancing
- You need encrypted inter-node communication

Multi-Arch Builds (ARM64 + AMD64)

```
# Create a builder that supports multiple platforms
docker buildx create --name multiarch --use

# Build and push for both architectures
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  --tag your-registry.com/mcp-server:latest \
  --tag your-registry.com/mcp-server:$(git rev-parse --short
HEAD) \
  --push \
  .
```

In your Dockerfile, ensure you're not using architecture-specific binaries. The python:3.12-slim base image handles this automatically, but compiled dependencies (like uvloop, httptools) need to be installed fresh in each platform's build stage.

Common Docker Pitfalls

1. Zombie Processes

Uvicorn spawns worker processes. If the main process dies, workers become zombies. Use tini as PID 1:

```
RUN apt-get update && apt-get install -y tini && rm -rf /var/lib/
apt/lists/*
ENTRYPOINT ["tini", "--"]
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port",
"8000", "--workers", "4"]
```

2. Log Rotation

Without limits, container logs fill the disk:

```
services:
  mcp:
    logging:
      driver: json-file
```

```
options:
  max-size: "50m" # Rotate at 50MB
  max-file: "5"   # Keep 5 rotated files
```

3. Memory Limits and OOM Kills

Python is memory-hungry. Set realistic limits:

```
deploy:
  resources:
    limits:
      memory: 1G # Hard limit – OOM kill above this
    reservations:
      memory: 256M # Guaranteed minimum
```

Monitor with:

```
docker stats --format "table {{.Name}}\t{{.MemUsage}}\t{{.MemPerc}}"
```

4. Graceful Shutdown

Uvicorn needs SIGTERM to shut down gracefully (finish in-progress requests):

```
services:
  mcp:
    stop_grace_period: 30s # Give 30s to finish requests before
    SIGKILL
    stop_signal: SIGTERM
```

In your FastMCP server, handle the ASGI lifespan:

```
from contextlib import asynccontextmanager
from fastmcp import FastMCP
```

```
@asynccontextmanager
async def lifespan(app):
    # Startup: initialize connections
    print("Starting MCP server...")
    await redis_client.initialize()
    await db.connect()
    yield
    # Shutdown: clean up
    print("Shutting down MCP server...")
    await db.disconnect()
    await redis_client.close()
```

```
mcp = FastMCP("Production Server", lifespan=lifespan)
```

5. DNS Resolution Timing

Container DNS can be slow on first resolution. Redis/database connections that fail on startup:

```
services:
  mcp:
    depends_on:
      redis:
        condition: service_healthy # Wait for Redis to be
actually ready
```

And in your application, use retry logic:

```
import asyncio
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(5),
wait=wait_exponential(multiplier=1, max=10))
async def connect_redis():
    client = redis.from_url(REDIS_URL)
    await client.ping()
    return client
```

Chapter 5: Security Hardening Checklist

Ten security items. Each one covers the threat, how to verify you're vulnerable, the complete fix, and how to validate the fix works.

1. Path Traversal Prevention

Threat: MCP tools that accept file paths can be tricked into reading/writing outside their intended directory. A tool input of ../../etc/passwd or /etc/shadow leaks sensitive system files.

Verify you're vulnerable:

```
# test_path_traversal.py
import httpx

# Attempt to read outside the allowed directory
payload = {
    "jsonrpc": "2.0",
    "id": 1,
    "method": "tools/call",
    "params": {
        "name": "read_file",
```

```

        "arguments": {"path": "../../../etc/passwd"}
    }
}

r = httpx.post("http://localhost:8000/mcp", json=payload,
headers={"Authorization": "Bearer TOKEN"})
if "root:" in r.text:
    print("VULNERABLE: Path traversal successful")
else:
    print("OK: Path traversal blocked")

```

The fix:

```

# tools/file_tools.py
import os
from pathlib import Path
from fastmcp import FastMCP

ALLOWED_BASE = Path("/app/data").resolve()

def safe_path(user_path: str) -> Path:
    """
    Resolve a user-provided path and verify it's within the
    allowed base.
    Prevents path traversal attacks.
    """
    # Resolve all symlinks and relative components
    resolved = (ALLOWED_BASE / user_path).resolve()

    # Verify the resolved path is still under our base
    if not str(resolved).startswith(str(ALLOWED_BASE)):
        raise ValueError(f"Access denied: path '{user_path}'
resolves outside allowed directory")

    # Additional checks
    if resolved.is_symlink():
        # Resolve symlink target and check again
        target = resolved.readlink().resolve()
        if not str(target).startswith(str(ALLOWED_BASE)):
            raise ValueError(f"Access denied: symlink target is
outside allowed directory")

    return resolved

@mcp.tool()
async def read_file(path: str) -> str:
    """Read a file from the data directory."""
    safe = safe_path(path)
    return safe.read_text()

```

Verify the fix:

```
# Should fail with "Access denied"
curl -X POST http://localhost:8000/mcp \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer TOKEN" \
  -d '{"jsonrpc": "2.0", "id": 1, "method": "tools/call", "params": {"name": "read_file", "arguments": {"path": "../../etc/passwd"}}}'
```

2. Rate Limiting

Threat: An attacker (or misconfigured client) floods your MCP server with requests, exhausting CPU, memory, or database connections. LLM-powered tools are especially expensive to invoke.

Verify you're vulnerable:

```
# Send 100 requests in 2 seconds
for i in $(seq 1 100); do
  curl -s -o /dev/null -w "%{http_code}\n" \
    -X POST http://localhost:8000/mcp \
    -H "Content-Type: application/json" \
    -d '{"jsonrpc": "2.0", "id": 1, "method": "tools/call", "params": {"name": "expensive_tool", "arguments": {}}}' &
done
wait

# If all return 200: no rate limiting in place
```

The fix:

```
# middleware/rate_limit.py
import time
import asyncio
from collections import defaultdict
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import JSONResponse

class TokenBucketRateLimiter(BaseHTTPMiddleware):
    """
    Per-client token bucket rate limiter.
    Keys by authenticated client_id (falls back to IP).
    """

    def __init__(self, app, rate: float = 10.0, burst: int = 20):
        super().__init__(app)
        self.rate = rate # Tokens per second
        self.burst = burst # Maximum bucket size
        self.buckets: dict[str, tuple[float, float]] = {} # key -
```

```

> (tokens, last_time)
    self._lock = asyncio.Lock()

    def _get_key(self, request: Request) -> str:
        # Use authenticated client_id if available
        auth_context = getattr(request.state, "auth", None)
        if auth_context and "client_id" in auth_context:
            return f"client:{auth_context['client_id']}"
        # Fall back to IP
        return f"ip:{request.client.host}"

    async def dispatch(self, request: Request, call_next):
        # Skip rate limiting for health checks
        if request.url.path in ("/health", "/liveness"):
            return await call_next(request)

        key = self._get_key(request)

        async with self._lock:
            now = time.monotonic()
            tokens, last_time = self.buckets.get(key,
            (self.burst, now))

            # Add tokens based on elapsed time
            elapsed = now - last_time
            tokens = min(self.burst, tokens + elapsed * self.rate)

            if tokens < 1:
                # Calculate retry-after
                retry_after = (1 - tokens) / self.rate
                return JSONResponse(
                    {"error": "rate_limit_exceeded",
                    "retry_after": round(retry_after, 1)},
                    status_code=429,
                    headers={"Retry-After": str(int(retry_after)
                    + 1)},
                )

            # Consume one token
            tokens -= 1
            self.buckets[key] = (tokens, now)

        return await call_next(request)

```

Verify the fix:

```

# Rapid requests should eventually return 429
for i in $(seq 1 30); do
    code=$(curl -s -o /dev/null -w "%{http_code}" -X POST http://
localhost:8000/mcp \
    -H "Content-Type: application/json" \
    -d '{"jsonrpc":"2.0","id":1,"method":"ping"}')

```

```

        echo "Request $i: $code"
    done
    # Should see 429 after burst limit

```

3. Input Sanitization

Threat: Tool arguments passed directly to shell commands, SQL queries, or file operations enable injection attacks.

Verify you're vulnerable:

```

# If your tool does this, you're vulnerable:
@mcp.tool()
async def search_logs(query: str) -> str:
    import subprocess
    result = subprocess.run(f"grep '{query}' /var/log/app.log",
shell=True, capture_output=True)
    return result.stdout.decode()

# Attack: query = "; rm -rf / #"

```

The fix:

```

# tools/safe_tools.py
import shlex
import subprocess
import re
from typing import Annotated
from pydantic import Field, field_validator
from fastmcp import FastMCP

# Pattern 1: Use Pydantic validation on tool inputs
class SearchQuery:
    query: Annotated[str, Field(max_length=200, pattern=r"^[a-zA-Z0-9\s._-]+$")]

@mcp.tool()
async def search_logs(query: str) -> str:
    """Search application logs for a pattern."""
    # Validate: only allow safe characters
    if not re.match(r"^[a-zA-Z0-9\s._-]+$", query):
        raise ValueError("Query contains invalid characters. Only alphanumeric, spaces, dots, underscores, and hyphens allowed.")

    if len(query) > 200:
        raise ValueError("Query too long (max 200 characters)")

    # Never use shell=True. Pass args as a list.
    result = subprocess.run(
        ["grep", "--", query, "/var/log/app.log"],

```

```

        capture_output=True,
        text=True,
        timeout=10,
    )
    return result.stdout[:10000] # Limit output size

# Pattern 2: For SQL, always use parameterized queries
@mcp.tool()
async def query_users(name: str) -> list[dict]:
    """Search users by name."""
    async with db.acquire() as conn:
        # NEVER: f"SELECT * FROM users WHERE name = '{name}'"
        # ALWAYS: parameterized
        rows = await conn.fetch(
            "SELECT id, name, email FROM users WHERE name ILIKE $1 LIMIT 50",
            f"%{name}%",
        )
    return [dict(row) for row in rows]

```

Verify the fix:

```

# Attempt shell injection – should fail gracefully
curl -X POST http://localhost:8000/mcp \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer TOKEN" \
  -d '{"jsonrpc": "2.0", "id": 1, "method": "tools/call", "params": {
    "name": "search_logs", "arguments": {"query": "test; rm -rf /"} } }'
# Should return error about invalid characters, not execute the
command

```

4. CORS Hardening

Threat: A malicious website tricks a user's browser into making authenticated MCP requests, stealing data or executing tools on behalf of the user.

Verify you're vulnerable:

```

# Check if wildcard origin is allowed
curl -I -X OPTIONS https://mcp.yourdomain.com/mcp \
  -H "Origin: https://evil-site.com" \
  -H "Access-Control-Request-Method: POST"

# If Access-Control-Allow-Origin: * appears, you're vulnerable

```

The fix: See the complete CORS configuration in Chapter 2. Key points:

1. Never use Access-Control-Allow-Origin: * in production

2. Whitelist specific origins (Claude, Cursor, your app)
3. Include mcp-session-id and mcp-protocol-version in allowed/exposed headers
4. Use the always directive on all CORS headers

5. TLS Configuration

Threat: Weak TLS allows man-in-the-middle attacks, capturing auth tokens and tool responses in transit.

Verify you're vulnerable:

```
# Check for TLS 1.0/1.1 support (should be disabled)
openssl s_client -connect mcp.yourdomain.com:443 -tls1 < /dev/
null 2>&1 | grep "Protocol"
openssl s_client -connect mcp.yourdomain.com:443 -tls1_1 < /dev/
null 2>&1 | grep "Protocol"
# If either connects successfully, you're vulnerable

# Check for weak ciphers
nmap --script ssl-enum-ciphers -p 443 mcp.yourdomain.com | grep -
E "(weak|MEDIUM)"
```

The fix: The nginx config in Chapter 2 already implements this.
Verify with:

```
# Online: SSL Labs test
# https://www.ssllabs.com/ssltest/analyze.html?
d=mcp.yourdomain.com

# CLI verification
testssl.sh mcp.yourdomain.com
```

6. Secrets Management

Threat: Hardcoded secrets in source code, Docker images, or environment variables get leaked through git history, image registries, or docker inspect.

Verify you're vulnerable:

```
# Check for secrets in git history
git log --all -p | grep -iE "(password|secret|api_key|token)" |
head -20

# Check Docker image layers
docker history your-mcp-server:latest --no-trunc | grep -iE "(ENV|
ARG).*(password|secret|key)"
```

```
# Check running container environment
docker inspect mcp-server-1 | jq '.[0].Config.Env'
```

The fix: Use Docker Secrets (see Chapter 4) or an external vault.
For the application layer:

```
# Never do this:
API_KEY = "sk-1234567890abcdef" # Hardcoded

# Do this instead:
API_KEY = get_secret("api_key") # Loaded from /run/secrets/ or
vault
```

For git, add a pre-commit hook:

```
#!/bin/bash
# .git/hooks/pre-commit
if git diff --cached --name-only | xargs grep -lE "(sk-[a-zA-Z0-9]{20,}|password\s*=\s*['\"](?:[^\"]|'')*)" 2>/dev/null; then
    echo "ERROR: Potential secret detected in staged files"
    exit 1
fi
```

7. Non-Root Container

Threat: A container running as root that gets compromised gives the attacker root access to the host (especially with misconfigured volume mounts or capability escalation).

Verify you're vulnerable:

```
docker exec mcp-server-1 whoami
# If it says "root", you're vulnerable

docker exec mcp-server-1 id
# uid=0(root) = bad
```

The fix: Already covered in the Dockerfile (Chapter 4).
Additionally:

```
# docker-compose.yml
services:
  mcp:
    security_opt:
      - no-new-privileges:true # Prevent privilege escalation
    read_only: true # Read-only filesystem
    tmpfs:
      - /tmp # Writable temp directory
    cap_drop:
      - ALL # Drop all Linux capabilities
```

```
cap_add:
  - NET_BIND_SERVICE # Only if binding to ports < 1024
```

Verify the fix:

```
docker exec mcp-server-1 id
# Should show: uid=1000(mcp) gid=1000(mcp)

# Try to write to filesystem (should fail)
docker exec mcp-server-1 touch /test
# touch: cannot touch '/test': Read-only file system
```

8. Tool-Level Access Control

Threat: All authenticated users can access all tools, regardless of their role. An intern with MCP access can trigger production deployments.

The fix:

```
# auth/tool_acl.py
from functools import wraps
from fastmcp import FastMCP

# Tool access control list
TOOL_ACL = {
    "query_database": ["analyst", "engineer", "admin"],
    "deploy_service": ["engineer", "admin"],
    "delete_data": ["admin"],
    "read_file": ["analyst", "engineer", "admin"],
}

def check_tool_access(tool_name: str, user_roles: list[str]) ->
bool:
    """Check if user has access to a specific tool."""
    allowed_roles = TOOL_ACL.get(tool_name, [])
    if not allowed_roles:
        return False # Deny by default if tool not in ACL
    return bool(set(user_roles) & set(allowed_roles))

def require_role(*roles: str):
    """Decorator to restrict tool access by role."""
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            from fastmcp.server.dependencies import import
get_auth_context
            auth = get_auth_context()
            user_roles = auth.get("roles", [])
```

```

        if not set(roles) & set(user_roles):
            raise PermissionError(
                f"Access denied. Required role: {' or "
'.join(roles)}.'. "
                f"Your roles: {' , '.join(user_roles) or
'none'}"
            )
        return await func(*args, **kwargs)
    return wrapper
return decorator

@mcp.tool()
@require_role("engineer", "admin")
async def deploy_service(service: str, version: str) -> str:
    """Deploy a service to production."""
    # ... deployment logic ...
    return f"Deployed {service}:{version}"

```

9. RBAC with Cerbos/OPA

Threat: Hardcoded ACLs become unmaintainable at scale. Policy changes require code deployments.

The fix with Cerbos (policy-as-code engine):

```

# policies/mcp_tools.yaml (Cerbos policy)
apiVersion: api.cerbos.dev/v1
resourcePolicy:
  version: "1.0"
  resource: "mcp_tool"
  rules:
    - actions: ["execute"]
      roles: ["admin"]
      effect: EFFECT_ALLOW
      condition:
        match:
          all:
            of:
              - expr: "true" # Admins can execute any tool

    - actions: ["execute"]
      roles: ["engineer"]
      effect: EFFECT_ALLOW
      condition:
        match:
          all:
            of:
              - expr: 'request.resource.attr.category !=
"destructive"'

```

```

        - actions: ["execute"]
        roles: ["analyst"]
        effect: EFFECT_ALLOW
        condition:
            match:
                all:
                    of:
                        - expr: 'request.resource.attr.category ==
"read_only"'

# auth/cerbos_check.py
import httpx
from functools import wraps

CERBOS_URL = "http://cerbos:3592"

    async def check_permission(user_id: str, roles: list[str],
tool_name: str, tool_category: str) -> bool:
    """Check tool execution permission with Cerbos."""
    payload = {
        "principal": {
            "id": user_id,
            "roles": roles,
        },
        "resources": [
            {
                "resource": {
                    "kind": "mcp_tool",
                    "id": tool_name,
                    "attr": {"category": tool_category},
                },
                "actions": ["execute"],
            }
        ],
    }

    async with httpx.AsyncClient() as client:
        resp = await client.post(f"{CERBOS_URL}/api/check",
json=payload)
        result = resp.json()

    return result["results"][0]["actions"]["execute"] ==
"EFFECT_ALLOW"

```

10. Audit Logging

Threat: When something goes wrong (data leak, unauthorized access, tool misuse), you have no trail of who did what.

The fix:

```
# middleware/audit.py
import json
import time
import uuid
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
import structlog

logger = structlog.get_logger("audit")

class AuditMiddleware(BaseHTTPMiddleware):
    """Log every MCP tool invocation with full context."""

    async def dispatch(self, request: Request, call_next):
        # Skip non-MCP requests
        if not request.url.path.startswith("/mcp"):
            return await call_next(request)

        request_id = str(uuid.uuid4())
        start_time = time.time()

        # Extract auth context
        auth = getattr(request.state, "auth", {})

        # Read request body for tool calls
        body = await request.body()
        try:
            parsed = json.loads(body) if body else {}
        except json.JSONDecodeError:
            parsed = {}

        # Execute request
        response = await call_next(request)

        # Log the audit event
        duration_ms = (time.time() - start_time) * 1000

        audit_event = {
            "request_id": request_id,
            "timestamp": time.time(),
            "user": auth.get("sub", "anonymous"),
            "client_id": auth.get("client_id", "unknown"),
            "ip": request.client.host,
            "method": parsed.get("method"),
            "tool": parsed.get("params", {}).get("name"),
            "arguments": parsed.get("params",
                {}).get("arguments"),
            "status_code": response.status_code,
            "duration_ms": round(duration_ms, 2),
        }
```

```

        "session_id": request.headers.get("mcp-session-id"),
    }

    if response.status_code >= 400:
        logger.warning("mcp_request_failed", **audit_event)
    else:
        logger.info("mcp_request", **audit_event)

    return response

```

Bonus: WAF Rules for MCP Endpoints

Using Coraza (open-source WAF compatible with ModSecurity rules):

```

# /etc/coraza/mcp-rules.conf

# Block requests with suspicious tool arguments
SecRule ARGS:arguments "@rx (\\.\\.\\.|/etc/|/proc/|;|\\|'|`)" \
    "id:100001,phase:2,deny,status:403,msg:'Path traversal in tool arguments'"

# Block oversized JSON bodies (DoS prevention)
SecRule REQUEST_BODY_LENGTH "@gt 1048576" \
    "id:100002,phase:1,deny,status:413,msg:'Request body too large (>1MB)'"

# Rate limit tool calls per session
SecRule &TX:mcp_tool_calls "@gt 100" \
    "id:100003,phase:1,deny,status:429,msg:'Too many tool calls in session'"

```

Bonus: Security Scanning in CI

```

# .github/workflows/security.yml
name: Security Scan
on: [push, pull_request]

jobs:
  scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Python dependency vulnerabilities
      - name: Run Bandit (Python SAST)
        run: |
          pip install bandit
          bandit -r . -f json -o bandit-report.json || true

      # Docker image vulnerabilities

```

```

- name: Build image for scanning
  run: docker build -t mcp-scan:latest .

- name: Run Trivy vulnerability scanner
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: mcp-scan:latest
    format: table
    exit-code: 1
    severity: CRITICAL,HIGH

# Dependency audit
- name: Run pip-audit
  run: |
    pip install pip-audit
    pip-audit -r requirements.txt --fix --dry-run

```

Chapter 6: CI/CD Pipeline

Your MCP server needs automated testing, building, scanning, and deployment. This chapter gives you a complete GitHub Actions workflow that takes code from push to production with safety gates at every step.

Complete GitHub Actions Workflow

```

# .github/workflows/deploy.yml
name: Build, Test, Deploy MCP Server

on:
  push:
    branches: [main, staging]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }
  PYTHON_VERSION: "3.12"

jobs:
  # =====
  # Stage 1: Test
  # =====
  test:
    runs-on: ubuntu-latest
    services:
      redis:
        image: redis:7-alpine
        ports:

```

```

    - 6379:6379
options: >-
  --health-cmd "redis-cli ping"
  --health-interval 10s
  --health-timeout 5s
  --health-retries 5

steps:
  - uses: actions/checkout@v4

  - name: Set up Python
    uses: actions/setup-python@v5
    with:
      python-version: ${{ env.PYTHON_VERSION }}

  - name: Cache dependencies
    uses: actions/cache@v4
    with:
      path: ~/.cache/pip
      key: <equation>{{ runner.os }}-pip-</
equation>{{ hashFiles('requirements*.txt') }}
      restore-keys: ${{ runner.os }}-pip-

  - name: Install dependencies
    run: |
      pip install -r requirements.txt
      pip install -r requirements-dev.txt

  - name: Run linting
    run: |
      ruff check .
      ruff format --check .

  - name: Run type checking
    run: mypy server.py --ignore-missing-imports

  - name: Run tests with coverage
    env:
      REDIS_URL: redis://localhost:6379/0
      MCP_ENV: test
    run: |
      pytest tests/ \
        --cov=. \
        --cov-report=xml \
        --cov-report=term-missing \
        --cov-fail-under=80 \
        -v

  - name: Upload coverage
    uses: codecov/codecov-action@v4
    with:
      file: coverage.xml

```

```

        fail_ci_if_error: false

# =====
# Stage 2: Build + Scan
# =====
build:
  needs: test
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write
    security-events: write

  outputs:
    image_tag: ${ steps.meta.outputs.tags }
    image_digest: ${ steps.build.outputs.digest }

  steps:
    - uses: actions/checkout@v4

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Set up QEMU (for multi-arch)
      uses: docker/setup-qemu-action@v3

    - name: Login to Container Registry
      uses: docker/login-action@v3
      with:
        registry: ${ env.REGISTRY }
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }

    - name: Extract metadata
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: <equation>${ env.REGISTRY }/</equation>${ env.IMAGE_NAME }
        tags: |
          type=sha,prefix=
          type=ref,event=branch
          type=semver,pattern={{version}}
          type=raw,value=latest,enable=${ github.ref == 'refs/heads/main' }}

    - name: Build and push
      id: build
      uses: docker/build-push-action@v6
      with:
        context: .
        platforms: linux/amd64,linux/arm64

```

```

        push: true
        tags: ${ steps.meta.outputs.tags }
        labels: ${ steps.meta.outputs.labels }
        cache-from: type=gha
        cache-to: type=gha,mode=max

    - name: Run Trivy vulnerability scanner
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: <equation>{{ env.REGISTRY }}/</
equation>{{ env.IMAGE_NAME }}:${{ github.sha }}
        format: sarif
        output: trivy-results.sarif
        severity: CRITICAL,HIGH
        exit-code: 1

    - name: Upload Trivy scan results
      uses: github/codeql-action/upload-sarif@v3
      if: always()
      with:
        sarif_file: trivy-results.sarif

# =====
# Stage 3: Deploy to Staging
# =====
deploy-staging:
  needs: build
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main' || github.ref == 'refs/
heads/staging'
  environment:
    name: staging
    url: https://mcp-staging.yourdomain.com

  steps:
    - uses: actions/checkout@v4

    - name: Deploy to staging server
      uses: appleboy/ssh-action@v1
      with:
        host: ${ secrets.STAGING_HOST }
        username: deploy
        key: ${ secrets.STAGING_SSH_KEY }
        script: |
          cd /opt/mcp-server

          # Pull the new image
          docker pull <equation>{{ env.REGISTRY }}/</
equation>{{ env.IMAGE_NAME }}:${{ github.sha }}

          # Update the image tag in compose
          export IMAGE_TAG=${{ github.sha }}

```

```

# Rolling update with zero downtime
docker compose up -d --no-deps --scale mcp=2 mcp
sleep 10

# Health check
for i in $(seq 1 30); do
    if curl -sf http://localhost:8000/health | grep -q
'"healthy"'; then
        echo "Health check passed on attempt $i"
        break
    fi
    if [ $i -eq 30 ]; then
        echo "Health check failed after 30 attempts"
        docker compose logs mcp --tail 50
        exit 1
    fi
    sleep 2
done

# Scale back down
docker compose up -d --no-deps --scale mcp=1 mcp

# Clean up old images
docker image prune -f --filter "until=72h"

- name: Run smoke tests against staging
  run: |
    # Wait for deployment to stabilize
    sleep 5

    # Test OAuth metadata
    curl -sf https://mcp-staging.yourdomain.com/.well-known/
oauth-authorization-server | jq .issuer

    # Test health endpoint
    curl -sf https://mcp-staging.yourdomain.com/health |
jq .status

    # Test MCP endpoint responds (should be 401 without
auth)
    status=$(curl -s -o /dev/null -w "%{http_code}" -X POST
https://mcp-staging.yourdomain.com/mcp)
    if [ "$status" != "401" ]; then
        echo "Expected 401, got $status"
        exit 1
    fi

# =====
# Stage 4: Deploy to Production
# =====
deploy-production:

```

```

needs: [build, deploy-staging]
runs-on: ubuntu-latest
if: github.ref == 'refs/heads/main'
environment:
  name: production
  url: https://mcp.yourdomain.com

steps:
  - uses: actions/checkout@v4

  - name: Deploy to production (canary)
    uses: appleboy/ssh-action@v1
    with:
      host: ${ secrets.PRODUCTION_HOST }
      username: deploy
      key: ${ secrets.PRODUCTION_SSH_KEY }
      script: |
        cd /opt/mcp-server

        # Pull new image
        docker pull <equation>{{ env.REGISTRY }}/</
equation>{{ env.IMAGE_NAME }}:${{ github.sha }}

        # Save current image tag for rollback
        docker compose config | grep "image:" | head -1 | awk
'{{print $2}}' > .rollback-image

        # Deploy canary (1 of 4 replicas)
        export IMAGE_TAG=${{ github.sha }}
        docker compose up -d --no-deps --scale mcp=4 mcp

        # Wait for canary to be ready
        sleep 15

        # Health check
        for i in $(seq 1 20); do
          if curl -sf http://localhost:8000/health | grep -q
'healthy'; then
            echo "Canary health check passed"
            break
          fi
          if [ $i -eq 20 ]; then
            echo "Canary failed – rolling back"
            export IMAGE_TAG=$(cat .rollback-image | awk -F:
'{{print $2}}')

            docker compose up -d --no-deps mcp
            exit 1
          fi
          sleep 3
        done

  - name: Verify production deployment

```

```

run: |
    sleep 10

    # Full production smoke test
    curl -sf https://mcp.yourdomain.com/health | jq .

    # Verify response time is reasonable (<500ms)
    time_total=$(curl -s -o /dev/null -w "%{time_total}"
https://mcp.yourdomain.com/health)
    if (( $(echo "$time_total > 0.5" | bc -l) )); then
        echo "WARNING: Health check took ${time_total}s
(>500ms)"
    fi

# =====
# Rollback (manual trigger)
# =====
rollback:
    runs-on: ubuntu-latest
    if: github.event_name == 'workflow_dispatch'
    environment: production

    steps:
        - name: Rollback to previous version
          uses: appleboy/ssh-action@v1
          with:
            host: ${ secrets.PRODUCTION_HOST }
            username: deploy
            key: ${ secrets.PRODUCTION_SSH_KEY }
            script: |
                cd /opt/mcp-server

                # Get the rollback image
                ROLLBACK_IMAGE=$(cat .rollback-image)
                echo "Rolling back to: $ROLLBACK_IMAGE"

                # Deploy rollback image
                export IMAGE_TAG=$(echo $ROLLBACK_IMAGE | awk -F:
'{print $2}')

                docker compose up -d --no-deps mcp

                # Verify
                sleep 10
                curl -sf http://localhost:8000/health | jq .

```

Testing Stage: MCP-Specific Tests

```

# tests/test_mcp_integration.py
"""
Integration tests that verify MCP protocol compliance.
These tests connect as an MCP client and exercise the full

```

```

protocol.
"""
import pytest
import httpx
from fastmcp import Client

@pytest.fixture
async def mcp_client():
    """Create an MCP client connected to the test server."""
    async with Client("http://localhost:8000/mcp") as client:
        yield client

@pytest.mark.asyncio
async def test_tools_list(mcp_client):
    """Server should list all registered tools."""
    tools = await mcp_client.list_tools()
    assert len(tools) > 0
    # Verify required tool metadata
    for tool in tools:
        assert tool.name
        assert tool.description
        assert tool.inputSchema

@pytest.mark.asyncio
async def test_tool_execution(mcp_client):
    """Tool execution should return valid results."""
    result = await mcp_client.call_tool("ping", {})
    assert result is not None
    assert len(result) > 0

@pytest.mark.asyncio
async def test_tool_error_handling(mcp_client):
    """Tools should return structured errors, not crash the
server."""
    # Call with invalid arguments
    with pytest.raises(Exception) as exc_info:
        await mcp_client.call_tool("query_database", {"sql": ""})
    # Should be a tool error, not a protocol error
    assert "error" in str(exc_info.value).lower()

@pytest.mark.asyncio
async def test_session_persistence():
    """Session ID should be stable across requests."""
    async with httpx.AsyncClient() as http:
        # First request – should receive a session ID
        r1 = await http.post(
            "http://localhost:8000/mcp",

```

```

        json={"jsonrpc": "2.0", "id": 1, "method":
"initialize", "params": {"capabilities": {}}},
        headers={"mcp-protocol-version": "2025-11-05"},
    )
    session_id = r1.headers.get("mcp-session-id")
    assert session_id is not None

    # Second request with same session ID
    r2 = await http.post(
        "http://localhost:8000/mcp",
        json={"jsonrpc": "2.0", "id": 2, "method": "tools/
list"},

        headers={
            "mcp-protocol-version": "2025-11-05",
            "mcp-session-id": session_id,
        },
    )
    assert r2.status_code == 200

@pytest.mark.asyncio
async def test_auth_required():
    """Unauthenticated requests should be rejected."""
    async with httpx.AsyncClient() as http:
        r = await http.post(
            "http://localhost:8000/mcp",
            json={"jsonrpc": "2.0", "id": 1, "method": "tools/
list"},

        )
        assert r.status_code == 401

```

Environment Variable Management

```

# .env.staging
MCP_ENV=staging
LOG_LEVEL=debug
WORKERS=2
REDIS_URL=redis://redis:6379/0
CORS_ORIGINS=https://mcp-staging.yourdomain.com,https://claude.ai

# .env.production
MCP_ENV=production
LOG_LEVEL=info
WORKERS=4
REDIS_URL=redis://redis:6379/0
CORS_ORIGINS=https://mcp.yourdomain.com,https://claude.ai,https://
app.cursor.sh

```

Secrets (API keys, database passwords) go in GitHub Secrets, not .env files:

```
# In your workflow:
env:
  REDIS_URL: ${ secrets.REDIS_URL }
  DB_PASSWORD: ${ secrets.DB_PASSWORD }
  OAUTH_CLIENT_SECRET: ${ secrets.OAUTH_CLIENT_SECRET }
```

Common CI/CD Failures

Failure	Cause	Fix
pip install hangs	PyPI rate limiting on parallel CI runners	Use <code>pip install --retries 3 --timeout 60</code> and cache aggressively
Trivy finds CRITICAL in base image	Upstream Python image has unpatched CVE	Pin to a specific digest, or add <code>--ignore-unfixed</code> for known unfixable CVEs
Multi-arch build OOMs	ARM64 emulation via QEMU is memory-hungry	Use <code>runs-on: [self-hosted, arm64]</code> or increase runner memory
Deploy SSH timeout	Firewall blocks GitHub Actions IPs	Use a deploy key with IP allowlist, or tunnel via Tailscale/WireGuard
Health check passes then fails	Container starts before dependencies are ready	Add explicit <code>depends_on</code> with health conditions; use retry loops
Flaky tests in CI	Tests depend on timing or external services	Mock external calls; use <code>pytest-timeout</code> ; run flaky tests separately
Docker layer cache miss	<code>requirements.txt</code> changed, invalidating cache	Copy requirements first, install, then copy source code (already in our Dockerfile)
Race condition on deploy	Two pushes deploy simultaneously	Use GitHub's concurrency group to cancel in-progress deploys

Add concurrency control:

```
concurrency:
  group: deploy-${ github.ref }
  cancel-in-progress: true
```

Chapter 7: Monitoring & Alerting

You can't operate what you can't observe. This chapter sets up end-to-end observability for your MCP server: traces for debugging, metrics for alerting, logs for forensics, and dashboards for at-a-glance health.

OpenTelemetry Setup for FastMCP

FastMCP 3.x has built-in OpenTelemetry support. Enable it with minimal configuration:

```
# server.py
from fastmcp import FastMCP

mcp = FastMCP(
    "Production MCP Server",
    # Enable OpenTelemetry instrumentation
    instrumentation="otlp",
)
```

For full control over the OTEL pipeline:

```
# telemetry/setup.py
from opentelemetry import trace, metrics
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanExporter
from opentelemetry.sdk.metrics import MeterProvider
from opentelemetry.sdk.metrics.export import
PeriodicExportingMetricReader
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
OTLPSpanExporter
from opentelemetry.exporter.otlp.proto.grpc.metric_exporter
import OTLPMetricExporter
from opentelemetry.sdk.resources import Resource, SERVICE_NAME,
SERVICE_VERSION
from opentelemetry.instrumentation.httpx import
HTTPXClientInstrumentor
from opentelemetry.instrumentation.redis import RedisInstrumentor
import os

def setup_telemetry():
    """Initialize OpenTelemetry with OTLP export."""
    resource = Resource.create({
        SERVICE_NAME: "mcp-server",
        SERVICE_VERSION: os.getenv("GIT_SHA", "dev"),
        "deployment.environment": os.getenv("MCP_ENV",
"development"),
    })
```

```

        # Traces
        trace_exporter = OTLPSpanExporter(
            endpoint=os.getenv("OTEL_EXPORTER_OTLP_ENDPOINT", "http://
otel-collector:4317"),
        )
        trace_provider = TracerProvider(resource=resource)

        trace_provider.add_span_processor(BatchSpanExporter(trace_exporter))
        trace.set_tracer_provider(trace_provider)

        # Metrics
        metric_exporter = OTLPMetricExporter(
            endpoint=os.getenv("OTEL_EXPORTER_OTLP_ENDPOINT", "http://
otel-collector:4317"),
        )
        metric_reader
= PeriodicExportingMetricReader(metric_exporter,
    export_interval_millis=30000)
        meter_provider = MeterProvider(resource=resource,
metric_readers=[metric_reader])
        metrics.set_meter_provider(meter_provider)

        # Auto-instrument libraries
        HTTPXClientInstrumentor().instrument()
        RedisInstrumentor().instrument()

        return trace.get_tracer("mcp-server"), metrics.get_meter("mcp-
server")

tracer, meter = setup_telemetry()

```

Custom MCP Metrics

```

# telemetry/mcp_metrics.py
from opentelemetry import metrics
import time
from functools import wraps

meter = metrics.get_meter("mcp-server")

# Counters
tool_calls_total = meter.create_counter(
    "mcp.tool.calls.total",
    description="Total number of tool invocations",
    unit="1",
)

tool_errors_total = meter.create_counter(
    "mcp.tool.errors.total",
    description="Total number of tool execution errors",

```

```

        unit="1",
    )

    # Histograms
    tool_duration = meter.create_histogram(
        "mcp.tool.duration",
        description="Tool execution duration in seconds",
        unit="s",
    )

    request_size = meter.create_histogram(
        "mcp.request.size",
        description="Request body size in bytes",
        unit="By",
    )

    # Gauges (via UpDownCounter for OTEL)
    active_sessions = meter.create_up_down_counter(
        "mcp.sessions.active",
        description="Number of active MCP sessions",
        unit="1",
    )

    def instrument_tool(tool_name: str):
        """Decorator to add metrics to any MCP tool."""
        def decorator(func):
            @wraps(func)
            async def wrapper(*args, **kwargs):
                labels = {"tool": tool_name}
                tool_calls_total.add(1, labels)

                start = time.perf_counter()
                try:
                    result = await func(*args, **kwargs)
                    labels["status"] = "success"
                    return result
                except Exception as e:
                    labels["status"] = "error"
                    labels["error_type"] = type(e).__name__
                    tool_errors_total.add(1, labels)
                    raise
                finally:
                    duration = time.perf_counter() - start
                    tool_duration.record(duration, labels)

            return wrapper
        return decorator

    # Usage:
    @mcp.tool()

```

```

@instrument_tool("query_database")
async def query_database(sql: str) -> str:
    """Execute a SQL query."""
    # ...

```

Prometheus Metrics Endpoint

If you prefer Prometheus pull-based collection over OTLP push:

```

# telemetry/prometheus.py
from prometheus_client import Counter, Histogram, Gauge,
generate_latest, CONTENT_TYPE_LATEST
from starlette.requests import Request
from starlette.responses import Response

# Define metrics
TOOL_CALLS = Counter(
    "mcp_tool_calls_total",
    "Total MCP tool invocations",
    ["tool", "status"],
)

TOOL_DURATION = Histogram(
    "mcp_tool_duration_seconds",
    "Tool execution duration",
    ["tool"],
    buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0,
30.0, 60.0],
)

ACTIVE_SESSIONS = Gauge(
    "mcp_active_sessions",
    "Currently active MCP sessions",
)

REQUEST_SIZE = Histogram(
    "mcp_request_size_bytes",
    "Request body size",
    buckets=[100, 500, 1000, 5000, 10000, 50000, 100000],
)

async def metrics_endpoint(request: Request) -> Response:
    """Expose Prometheus metrics at /metrics."""
    return Response(
        generate_latest(),
        media_type=CONTENT_TYPE_LATEST,
    )

```

Key Metrics Dashboard

The metrics that matter for an MCP server:

Metric	Type	Alert Threshold	What It Tells You
mcp_tool_duration_seconds{quantile="0.95"}	Histogram	>5s	Tools are slow — check external deps
mcp_tool_calls_total{status="error"} / total	Rate	>5% error rate	Something is breaking
mcp_active_sessions	Gauge	>1000	Session leak or surge
mcp_tool_calls_total rate	Counter	<1 rpm for 10min	Traffic dropped — possible outage
mcp_request_size_bytes{quantile="0.99"}	Histogram	>1MB	Clients sending oversized payloads

Grafana Dashboard JSON

```
{
  "dashboard": {
    "title": "MCP Server Overview",
    "uid": "mcp-server-overview",
    "timezone": "browser",
    "panels": [
      {
        "title": "Tool Call Rate (per second)",
        "type": "timeseries",
        "gridPos": {"h": 8, "w": 12, "x": 0, "y": 0},
        "targets": [
          {
            "expr": "sum(rate(mcp_tool_calls_total[5m])) by
(tool)",
            "legendFormat": "{{tool}}"
          }
        ]
      },
      {
        "title": "Error Rate (%)",
        "type": "timeseries",
```

```

        "gridPos": {"h": 8, "w": 12, "x": 12, "y": 0},
        "targets": [
            {
                "expr":
"sum(rate(mcp_tool_calls_total{status=\"error\"}[5m])) /
sum(rate(mcp_tool_calls_total[5m])) * 100",
                "legendFormat": "Error %"
            }
        ]
    },
    {
        "title": "Latency Percentiles (seconds)",
        "type": "timeseries",
        "gridPos": {"h": 8, "w": 12, "x": 0, "y": 8},
        "targets": [
            {
                "expr": "histogram_quantile(0.50,
sum(rate(mcp_tool_duration_seconds_bucket[5m])) by (le))",
                "legendFormat": "p50"
            },
            {
                "expr": "histogram_quantile(0.95,
sum(rate(mcp_tool_duration_seconds_bucket[5m])) by (le))",
                "legendFormat": "p95"
            },
            {
                "expr": "histogram_quantile(0.99,
sum(rate(mcp_tool_duration_seconds_bucket[5m])) by (le))",
                "legendFormat": "p99"
            }
        ]
    },
    {
        "title": "Active Sessions",
        "type": "stat",
        "gridPos": {"h": 4, "w": 6, "x": 12, "y": 8},
        "targets": [
            {
                "expr": "mcp_active_sessions",
                "legendFormat": "Sessions"
            }
        ]
    },
    {
        "title": "Tool Calls by Status",
        "type": "piechart",
        "gridPos": {"h": 4, "w": 6, "x": 18, "y": 8},
        "targets": [
            {
                "expr": "sum(increase(mcp_tool_calls_total[1h])) by
(status)",
                "legendFormat": "{{status}}"
            }
        ]
    }
]

```

```

    }
  ]
},
{
  "title": "Top 10 Slowest Tools (p95)",
  "type": "bargauge",
  "gridPos": {"h": 8, "w": 24, "x": 0, "y": 16},
  "targets": [
    {
      "expr": "topk(10, histogram_quantile(0.95,
sum(rate(mcp_tool_duration_seconds_bucket[5m])) by (le, tool)))",
      "legendFormat": "{{tool}}"
    }
  ]
}
]
}
}

```

Alerting Rules

```

# prometheus/alerts/mcp.yml
groups:
- name: mcp-server
  rules:
    # High error rate
    - alert: MCPHighErrorRate
      expr: |
        sum(rate(mcp_tool_calls_total{status="error"}[5m]))
        / sum(rate(mcp_tool_calls_total[5m])) > 0.05
      for: 5m
      labels:
        severity: critical
        team: platform
      annotations:
        summary: "MCP server error rate > 5%"
        description: "Error rate is {{ $value |
humanizePercentage }} over the last 5 minutes"

    # High latency
    - alert: MCPHighLatency
      expr: |
        histogram_quantile(0.95,
sum(rate(mcp_tool_duration_seconds_bucket[5m])) by (le)) > 10
      for: 5m
      labels:
        severity: warning
        team: platform
      annotations:
        summary: "MCP p95 latency > 10s"
        description: "95th percentile latency is {{ $value }}s"

```

```

# No traffic (possible outage)
- alert: MCPNoTraffic
  expr: sum(rate(mcp_tool_calls_total[10m])) == 0
  for: 15m
  labels:
    severity: warning
    team: platform
  annotations:
    summary: "No MCP tool calls for 15 minutes"
    description: "The MCP server hasn't received any tool
calls. Possible outage or client disconnection."

# Session leak
- alert: MCPSessionLeak
  expr: mcp_active_sessions > 500
  for: 10m
  labels:
    severity: warning
  annotations:
    summary: "Unusually high active sessions ({{ $value }})"

# Health check failing
- alert: MCPHealthCheckFailing
  expr: probe_success{job="mcp-health"} == 0
  for: 2m
  labels:
    severity: critical
    team: platform
  annotations:
    summary: "MCP server health check failing"

```

Structured Logging with Loki

```

# telemetry/logging.py
import structlog
import logging
import sys
import os

def setup_logging():
    """Configure structured JSON logging for Loki ingestion."""

    # Configure structlog
    structlog.configure(
        processors=[
            structlog.contextvars.merge_contextvars,
            structlog.processors.add_log_level,
            structlog.processors.StackInfoRenderer(),
            structlog.dev.set_exc_info,

```

```

        structlog.processors.TimeStamper(fmt="iso"),
        # Add service metadata
        structlog.processors.CallsiteParameterAdder(
            [structlog.processors.CallsiteParameter.FUNC_NAME]
        ),
        # JSON output for production, pretty for development
        structlog.processors.JSONRenderer(),
        if os.getenv("MCP_ENV") == "production"
        else structlog.dev.ConsoleRenderer(),
    ],
    wrapper_class=structlog.make_filtering_bound_logger(
        logging.getLevelName(os.getenv("LOG_LEVEL",
"INFO")).upper())
    ),
    context_class=dict,
    logger_factory=structlog.PrintLoggerFactory(),
    cache_logger_on_first_use=True,
)

# Redirect standard library logging through structlog
logging.basicConfig(
    format="%(message)s",
    stream=sys.stdout,
    level=logging.getLevelName(os.getenv("LOG_LEVEL",
"INFO")).upper()),
)

# Usage in tools:
logger = structlog.get_logger()

@mcp.tool()
async def query_database(sql: str) -> str:
    log = logger.bind(tool="query_database", sql_preview=sql[:
100])

    log.info("tool_invoked")

    try:
        result = await db.execute(sql)
        log.info("tool_completed", row_count=len(result))
        return str(result)
    except Exception as e:
        log.error("tool_failed", error=str(e),
error_type=type(e).__name__)
        raise

```

Loki LogQL Queries

```

# All errors in the last hour
{service="mcp-server"} | json | level="error"

```

```
# Tool execution failures by type
{service="mcp-server"} | json | event="tool_failed" | line_format
"{{.tool}}: {{.error}}"

# Slow tool calls (>5s)
{service="mcp-server"} | json | event="tool_completed" | duration
> 5

# Auth failures
{service="mcp-server"} | json | status_code=401

# Requests from a specific client
{service="mcp-server"} | json | client_id="mcp_abc123"
```

Health Check Endpoints: Liveness vs Readiness

```
# For Kubernetes or Docker health checks:

# Liveness: "Is the process alive and not deadlocked?"
# If this fails, the container gets killed and restarted.
@app.route("/liveness")
async def liveness(request):
    return JsonResponse({"status": "alive"})

# Readiness: "Can this instance serve traffic?"
# If this fails, traffic is routed away but the container stays
alive.
@app.route("/health")
async def readiness(request):
    # Check all dependencies
    checks = await run_health_checks()
    if all(c["status"] == "ok" for c in checks.values()):
        return JsonResponse({"status": "healthy", "checks":
checks})
    return JsonResponse({"status": "unhealthy",
"checks": checks}, status_code=503)
```

SLI/SLO Definition

SLI (Indicator)	SLO (Objective)	Measurement
Availability	99.9% uptime/ month	probe_success{job="mcp-health"}
Latency	p95 < 5s for tool calls	histogram_quantile(0.95, mcp_tool_duration_seconds_bucket)
Error rate		

SLI (Indicator)	SLO (Objective)	Measurement
	< 1% of tool calls fail	mcp_tool_calls_total{status="error"} / total
Throughput	Handle 100 concurrent sessions	mcp_active_sessions + load test baseline

Error budget: At 99.9% availability, you get **43 minutes of downtime per month**. Track this:

```
error_budget_remaining = 1 - (
    sum(rate(mcp_tool_calls_total{status="error"}[30d]))
    / sum(rate(mcp_tool_calls_total[30d]))
) / (1 - 0.999)
```

Chapter 8: 30+ Production Errors & Fixes

Real error messages from production MCP deployments. Each one includes the exact log output, root cause analysis, why it's confusing, the fix, and how to prevent it from happening again.

CORS Errors

Error 1: Silent CORS Failure — No Error in Server Logs

Error message (browser console):

```
Access to fetch at 'https://mcp.example.com/mcp' from origin
'https://claude.ai'
has been blocked by CORS policy: No 'Access-Control-Allow-Origin'
header is present
on the requested resource.
```

Root cause: The server returns CORS headers only on 2xx responses. When the MCP server returns 401 (missing auth) or 500 (server error), nginx drops the CORS headers because add_header without always only fires on success responses.

Why it's confusing: The developer sees "CORS error" and spends hours tweaking CORS config, when the actual problem is an auth or server error being masked by missing CORS headers on error responses.

The fix:

```
# Add 'always' to EVERY CORS header
add_header Access-Control-Allow-Origin $cors_origin always;
add_header Access-Control-Expose-Headers "mcp-session-id" always;
```

Prevention: Always test CORS with intentionally bad requests (wrong auth, malformed body) to verify headers appear on error responses.

Error 2: CORS Preflight Passes, Actual Request Fails

Error message:

```
Access to fetch at 'https://mcp.example.com/mcp' from origin
'https://app.cursor.sh'
has been blocked by CORS policy: Request header field mcp-
protocol-version is not
allowed by Access-Control-Allow-Headers in preflight response.
```

Root cause: The Access-Control-Allow-Headers list doesn't include MCP-specific headers (mcp-protocol-version, mcp-session-id). The OPTIONS preflight passes (it doesn't send these headers), but the actual POST request with MCP headers gets blocked.

The fix:

```
add_header Access-Control-Allow-Headers "Authorization, Content-
Type, mcp-protocol-version, mcp-session-id" always;
```

Prevention: Include all MCP headers in your CORS config from day one. The MCP spec may add more headers — check the changelog on protocol updates.

Error 3: Credentials Mode Conflicts with Wildcard Origin

Error message:

```
Access to fetch at 'https://mcp.example.com/mcp' from origin
'https://claude.ai'
has been blocked by CORS policy: The value of the 'Access-Control-
Allow-Origin'
header in the response must not be the wildcard '*' when the
request's credentials
mode is 'include'.
```

Root cause: The client sends credentials: "include" (for cookies or auth headers), but the server responds with Access-Control-Allow-Origin: *. Browsers refuse this combination as a security measure.

The fix: Return the specific requesting origin instead of *:

```
# Use the map-based origin matching from Chapter 2
add_header Access-Control-Allow-Origin $cors_origin always;
# NOT: add_header Access-Control-Allow-Origin * always;
```

Transport and Connection Errors

Error 4: Transport Inference Failure

Error message (FastMCP client):

```
fastmcp.exceptions.TransportError: Unable to infer transport for
'mcp.example.com'.
Provide a full URL with scheme (http:// or https://) or a file
path.
```

Root cause: The client was given just a hostname without a scheme. FastMCP 3.x needs a full URL to determine whether to use HTTP transport.

The fix: Always use the full URL:

```
# Wrong
client = Client("mcp.example.com/mcp")

# Right
client = Client("https://mcp.example.com/mcp")
```

Error 5: SSE Connection Drops After 60 Seconds

Error message (client logs):

```
httpx.ReadTimeout: timed out
Connection to MCP server lost after 60.0s
```

Root cause: If you're still using SSE transport (legacy), nginx's default `proxy_read_timeout` is 60s. SSE connections are long-lived — they get killed when idle.

The fix: Increase timeout or switch to Streamable HTTP:

```
# For SSE (legacy):
proxy_read_timeout 3600s;

# Better: switch to Streamable HTTP (FastMCP 3.x default)
# No long-lived connections needed
```

Error 6: "Connection refused" After Docker Restart

Error message:

httpx.ConnectError: [Errno 111] Connection refused

Root cause: The MCP container restarted but the client is still trying to connect to the old container's IP. Docker's internal DNS hasn't updated, or the client cached the connection.

The fix: Use service names (not IPs) in Docker networking, and implement retry with backoff:

```
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(5),
wait=wait_exponential(multiplier=1, max=10))
async def connect_mcp():
    return await Client("http://mcp:8000/mcp").connect()
```

Session Management Errors

Error 7: "Session not found" on Every Other Request

Error message:

```
{"jsonrpc": "2.0", "error": {"code": -32600, "message": "Session not found: abc123"}, "id": 1}
```

Root cause: Running uvicorn with multiple workers (--workers 4) but sessions are stored in-memory. Request 1 hits worker A (creates session), request 2 hits worker B (doesn't have the session).

The fix: Use Redis for session storage (see Chapter 3) or run with --workers 1 (not recommended for production).

Error 8: Session ID Not Returned to Client

Error message (client side):

TypeError: Cannot read properties of undefined (reading 'mcp-session-id')

Root cause: nginx isn't passing the mcp-session-id response header through, or it's missing from Access-Control-Expose-Headers so the browser hides it from JavaScript.

The fix:

```
proxy_pass_header mcp-session-id;  
add_header Access-Control-Expose-Headers "mcp-session-id" always;
```

Error 9: Session Memory Leak — Server OOM After 24 Hours

Error message (system):

```
Killed process 12345 (uvicorn) total-vm:4096000kB, anon-rss:  
2048000kB
```

Root cause: Sessions are created but never cleaned up. Each session holds tool state, context, and subscriptions. Without TTL, they accumulate until the process is OOM-killed.

The fix: Implement session expiry:

```
# In your session store:  
await redis_client.expire(f"mcp:session:{session_id}", 14400)  
# 4 hour TTL  
  
# Or with in-memory store, run periodic cleanup:  
import asyncio  
  
async def cleanup_sessions():  
    while True:  
        await asyncio.sleep(300) # Every 5 minutes  
        now = time.time()  
        expired = [sid for sid, s in sessions.items() if now -  
s.last_active > 14400]  
        for sid in expired:  
            del sessions[sid]
```

Authentication Errors

Error 10: Token Expiry Causes Cascading Failures

Error message:

```
401 Unauthorized: Token expired
```

Followed by hundreds of the same error as the client retries without refreshing.

Root cause: The MCP client doesn't handle 401 responses by refreshing the token. It just retries with the same expired token.

The fix: Implement token refresh on 401 (see TokenManager in Chapter 3). The client should:

1. Catch 401
 2. Refresh the access token
 3. Retry the original request once
 4. If still 401, re-authenticate from scratch
-

Error 11: PKCE Verification Fails Intermittently

Error message:

```
{"error": "invalid_grant", "error_description":  
"PKCE verification failed"}
```

Root cause: The code_verifier contains URL-safe base64 characters, but some HTTP libraries double-encode the + character when sending it in a form body.

The fix: Use secrets.token_urlsafe() for code verifiers (no + or / characters):

```
code_verifier = secrets.token_urlsafe(32) # Only uses [A-Za-  
z0-9_-]
```

Docker and Networking Errors

Error 12: Docker DNS Resolution Failure on Startup

Error message:

```
redis.exceptions.ConnectionError: Error -2 connecting to redis:  
6379. Name or service not known.
```

Root cause: The MCP container starts before Docker's internal DNS is ready, or before the Redis container has registered its DNS name.

The fix:

```
services:  
  mcp:  
    depends_on:  
      redis:  
        condition: service_healthy
```

Plus retry logic in the application (see Chapter 4).

Error 13: "Connection reset by peer" Under Load

Error message:

httpx.RemoteProtocolError: peer closed connection without sending complete message body

Root cause: Docker's default network bridge has a connection limit. Under high concurrent connections, Linux's `net.core.somaxconn` (default 128) or `net.ipv4.tcp_max_syn_backlog` causes connection resets.

The fix:

```
# docker-compose.yml
services:
  mcp:
    sysctls:
      - net.core.somaxconn=65535
      - net.ipv4.tcp_max_syn_backlog=65535
```

nginx and Proxy Errors

Error 14: "upstream sent too big header while reading response header"

Error message (nginx error log):

upstream sent too big header while reading response header from upstream

Root cause: The MCP server returns large headers (common with JWTs in Set-Cookie, or verbose error messages in custom headers). nginx's default `proxy_buffer_size` (4k or 8k) can't hold them.

The fix:

```
proxy_buffer_size 16k;
proxy_buffers 4 32k;
proxy_busy_buffers_size 64k;
```

Error 15: nginx Buffers SSE/Streaming — Client Sees Nothing for Minutes

Error message: No error — the client just hangs waiting for data.

Root cause: nginx's proxy_buffering on (the default) accumulates the streaming response in a buffer. The client doesn't see any data until the buffer fills or the connection closes.

The fix:

```
location /mcp {
    proxy_buffering off;
    proxy_cache off;
}
```

Error 16: 502 Bad Gateway After Deploy

Error message:

502 Bad Gateway

nginx error log:

```
connect() failed (111: Connection refused) while connecting to
upstream
```

Root cause: During deployment, the old container is stopped before the new one is ready. nginx tries to forward to a container that no longer exists.

The fix: Use rolling deployments (deploy new container → health check → remove old):

```
docker compose up -d --no-deps --scale mcp=2 mcp
sleep 10 # Wait for new container to pass health check
docker compose up -d --no-deps --scale mcp=1 mcp
```

Redis and State Errors

Error 17: Redis Connection Pool Exhausted

Error message:

```
redis.exceptions.ConnectionError: Error while reading from socket:
Connection reset by peer
redis.asyncio.connection.ConnectionPool: Max connections reached
```

Root cause: Each uvicorn worker opens its own Redis connection pool. With 4 workers × 10 connections per pool = 40 connections. Redis default max is 10,000, but your Redis instance might have a lower limit set.

The fix:

```
redis_client = redis.from_url(
    REDIS_URL,
    max_connections=20, # Per worker
    socket_connect_timeout=5,
    socket_timeout=5,
    retry_on_timeout=True,
)
```

And in Redis config:

```
maxclients 256
```

Error 18: Redis MEMORY Limit Reached — Silent Data Loss

Error message (Redis log):

```
# WARNING: Redis is configured to save RDB snapshots, but is
currently unable to persist on disk.
OOM command not allowed when used memory > 'maxmemory'
```

Root cause: Session data fills Redis beyond maxmemory. With allkeys-lru eviction policy, older sessions get evicted silently — users lose their MCP sessions without any error.

The fix: Monitor Redis memory and set appropriate limits:

```
maxmemory 256mb
maxmemory-policy allkeys-lru
```

Alert when memory usage exceeds 80%:

```
- alert: RedisMemoryHigh
  expr: redis_memory_used_bytes / redis_memory_max_bytes > 0.8
  for: 5m
```

Application Errors

Error 19: Starlette Middleware Ordering — Auth Bypassed

Error message: None — requests succeed without authentication.

Root cause: Middleware is applied in reverse order in Starlette/FastAPI. If you add AuthMiddleware before CORSMiddleware, CORS preflight (OPTIONS) requests hit auth first and get blocked. So you move auth after CORS — but now the order means auth doesn't fire at all for some paths.

The fix: Use FastMCP's built-in auth system instead of custom middleware, or be explicit about ordering:

```
# Middleware executes from BOTTOM to TOP in Starlette
app.add_middleware(AuthMiddleware)
# Executes 2nd (after CORS lets preflight through)
app.add_middleware(CORSMiddleware, ...) # Executes 1st
```

Error 20: ASGI Lifespan Event Not Firing

Error message:

RuntimeError: Redis connection not initialized

(on first request, despite lifespan setup)

Root cause: The lifespan context manager has an error that's silently swallowed. Or uvicorn was started without lifespan support (`--lifespan off`, which is the default in some configs).

The fix:

```
# Ensure lifespan is enabled
uvicorn server:app --lifespan on
```

And add error handling to lifespan:

```
@asynccontextmanager
async def lifespan(app):
    try:
        await initialize_connections()
        yield
    except Exception as e:
        print(f"LIFESPAN ERROR: {e}", file=sys.stderr)
        raise # Don't swallow – let it crash visibly
    finally:
        await cleanup_connections()
```

Error 21: Uvicorn Worker Timeout — Tools That Take Too Long

Error message:

WORKER TIMEOUT (pid: 12345)

Root cause: Uvicorn's default worker timeout (if using Gunicorn as process manager) is 30 seconds. Long-running MCP tools exceed this, and the worker gets killed mid-execution.

The fix:

```
# If using gunicorn + uvicorn workers:
gunicorn server:app -w 4 -k uvicorn.workers.UvicornWorker --
timeout 600

# If using uvicorn directly (no timeout issue – uvicorn doesn't
have worker timeout)
uvicorn server:app --workers 4
```

Error 22: Health Check False Positives — Server "Healthy" But Can't Serve

Error message: Health check returns 200 but clients get 500 errors.

Root cause: The health check only verifies HTTP connectivity (return {"status": "ok"}). It doesn't check whether the MCP tool registry is loaded, Redis is reachable, or database connections work.

The fix: Implement comprehensive health checks (see Chapter 4 health check implementation that verifies tools, Redis, and database).

Error 23: Rate Limiter Blocks Legitimate Batch Operations

Error message:

```
{"error": "rate_limit_exceeded", "retry_after": 2.5}
```

Root cause: An MCP client running a batch operation (e.g., "analyze these 50 files") hits the per-IP rate limit legitimately. The rate limiter doesn't distinguish between one client hammering and one client doing legitimate batch work.

The fix: Rate limit by authenticated client_id with higher limits for known clients:

```
# Different limits for different client tiers
RATE_LIMITS = {
    "premium": (50, 100), # 50 r/s, burst 100
    "standard": (10, 20), # 10 r/s, burst 20
    "anonymous": (2, 5), # 2 r/s, burst 5
}
```

TLS and Certificate Errors

Error 24: TLS Certificate Mismatch — Works in Browser, Fails in Code

Error message (Python/httpx):

```
ssl.SSLCertVerificationError: [SSL: CERTIFICATE_VERIFY_FAILED]
certificate verify failed:
Hostname mismatch, certificate is not valid for 'mcp.example.com'.
```

Root cause: The certificate covers example.com and www.example.com but not mcp.example.com. Browsers show a warning you can click through; programmatic clients (correctly) refuse.

The fix: Get a certificate that covers your MCP subdomain:

```
certbot --nginx -d example.com -d mcp.example.com -d
www.example.com
# Or use a wildcard:
certbot --nginx -d example.com -d "*.example.com" --preferred-
challenges dns
```

Error 25: Let's Encrypt Rate Limit Hit

Error message:

```
Error creating new order :: too many certificates (5) already
issued for this exact set of domains
```

Root cause: Failed certbot runs that actually issued certificates count toward the limit (5 per week for the same exact domain set).

The fix: Use staging for testing, production only for real certs:

```
# Test with staging first (no rate limits)
certbot --staging --nginx -d mcp.example.com

# Only after staging works:
certbot --nginx -d mcp.example.com
```

Protocol Errors

Error 26: "Method not found" for tools/call

Error message:

```
    {"jsonrpc": "2.0", "error": {"code": -32601, "message": "Method
not found: tools/call"}, "id": 1}
```

Root cause: The client is sending requests before completing the initialize handshake. The MCP protocol requires initialize → initialized before any other method.

The fix: Ensure your client completes initialization:

```
# FastMCP client does this automatically
async with Client("https://mcp.example.com/mcp") as client:
    # initialize + initialized happen in __aenter__
    tools = await client.list_tools() # Now this works
```

Error 27: JSON-RPC Batch Request Rejected

Error message:

```
    {"jsonrpc": "2.0", "error": {"code": -32600, "message": "Batch
requests not supported"}, "id": null}
```

Root cause: Some MCP clients try to batch multiple JSON-RPC calls in a single HTTP request (valid in JSON-RPC 2.0 spec). Not all MCP server implementations support this.

The fix: FastMCP 3.x supports batching out of the box. If you're on an older version, upgrade. If using a custom implementation, handle arrays:

```
async def handle_request(body):
    if isinstance(body, list):
        return [await process_single(req) for req in body]
    return await process_single(body)
```

Error 28: Protocol Version Mismatch

Error message:

```
    {"jsonrpc": "2.0", "error": {"code": -32600, "message":
"Unsupported protocol version: 2024-11-05. Supported:
2025-11-05"}, "id": 1}
```

Root cause: Client and server are on different MCP protocol versions. The server only supports the latest spec revision.

The fix: Update your client library, or configure the server to accept older versions:

```
mcp = FastMCP(
    "My Server",
    supported_protocol_versions=["2025-11-05", "2024-11-05"], #
```

Accept both

)

Kubernetes-Specific Errors

Error 29: K8s Readiness Probe Killing Healthy Pods During Tool Execution

Error message (kubectl events):

Readiness probe failed: HTTP probe failed with statuscode: 503

Root cause: During a long tool execution (30+ seconds), the readiness probe hits the health endpoint. If the health check is slow because all workers are busy with tool calls, K8s marks the pod as unready and stops sending traffic — creating a cascade.

The fix: Separate liveness (lightweight) from readiness (comprehensive), and increase probe tolerances:

```
livenessProbe:
  httpGet:
    path: /liveness
    port: 8000
  initialDelaySeconds: 10
  periodSeconds: 30
  timeoutSeconds: 5
  failureThreshold: 3

readinessProbe:
  httpGet:
    path: /health
    port: 8000
  initialDelaySeconds: 5
  periodSeconds: 10
  timeoutSeconds: 10 # Higher timeout for comprehensive checks
  failureThreshold: 3 # Don't flip on a single failure
```

Error 30: Pod Eviction During Long-Running Tool

Error message:

The node was low on resource: memory. Evicting container mcp-server.

Root cause: A tool that processes large files or generates long outputs spikes memory usage. K8s evicts the pod to protect the node.

The fix: Set realistic resource requests/limits and implement tool-level memory guards:

```
@mcp.tool()
async def process_file(file_path: str) -> str:
    """Process a file with memory safety."""
    import resource

    # Check memory before processing
    mem_mb = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss /
1024

    if mem_mb > 800: # 800MB threshold
        raise RuntimeError("Server under memory pressure. Try
again later.")

    # Stream processing instead of loading entire file
    # ...
```

Error 31: Service Mesh (Istio) Breaks MCP Streaming

Error message: Tool calls work but progress notifications never arrive.

Root cause: Istio's sidecar proxy buffers HTTP responses by default, preventing chunked streaming from reaching the client in real-time.

The fix: Annotate the pod to disable buffering for MCP traffic:

```
metadata:
  annotations:
    sidecar.istio.io/inject: "true"
    # Disable response buffering
    traffic.sidecar.istio.io/excludeInboundPorts: ""
    proxy.istio.io/config: |
      proxyMetadata:
        DISABLE_RESPONSE_BUFFERING: "true"
```

Miscellaneous Production Errors

Error 32: Tool Returns None — Client Gets Empty Response

Error message (client):

Tool 'analyze_data' returned no content

Root cause: The tool function doesn't explicitly return a value (Python returns None implicitly). FastMCP serializes None as an empty response.

The fix: Always return a meaningful string from tools:

```
@mcp.tool()
async def analyze_data(data: str) -> str:
    result = await do_analysis(data)
    if not result:
        return "Analysis complete. No notable findings." #
    Explicit "nothing found"
    return result
```

Error 33: Unicode Encoding Error in Tool Output

Error message:

UnicodeEncodeError: 'utf-8' codec can't encode character '\ud800' in position 42: surrogates not allowed

Root cause: Tool output contains surrogate characters (common when reading binary files as text, or processing malformed database content). JSON serialization rejects surrogates.

The fix:

```
@mcp.tool()
async def read_data(path: str) -> str:
    content = Path(path).read_bytes()
    # Replace surrogates and other problematic characters
    return content.decode("utf-8", errors="replace")
```

Error 34: Concurrent Tool Calls Deadlock

Error message: Server stops responding. No errors in logs. CPU at 0%.

Root cause: Two tools both acquire async locks in different order, causing a classic deadlock. Tool A holds lock 1, waits for lock 2. Tool B holds lock 2, waits for lock 1.

The fix: Always acquire locks in the same global order:

```
# Define a global lock ordering
LOCK_ORDER = {"database": 1, "cache": 2, "filesystem": 3}

async def acquire_ordered(*lock_names):
    """Acquire multiple locks in consistent order to prevent
    deadlocks."""
    sorted_locks = sorted(lock_names, key=lambda n: LOCK_ORDER[n])
    for name in sorted_locks:
        await locks[name].acquire()
```

This chapter covers the most common production issues. As FastMCP evolves, new error patterns will emerge. Monitor your error logs, contribute fixes upstream, and keep this chapter updated with your own discoveries.

Appendix A: Corrections & Updates Based on Technical Review

This appendix documents critical corrections to the main content based on external resource verification (June 2026).

Correction 1: Version Pinning

This manual is based on FastMCP 3.4.x (latest release: v3.4.2, June 6, 2026). FastMCP iterates rapidly — verify compatibility with your installed version before copying configurations.

```
# Check your version
pip show fastmcp | grep Version
# Expected: 3.4.x

# Pin in requirements.txt
fastmcp>=3.4.0,<4.0.0
starlette>=1.0.1 # CVE-2026-48710 fix
uvicorn>=0.30.0
```

Correction 2: Transport — Streamable HTTP is the Standard

FastMCP 3.x uses **Streamable HTTP** as the default and recommended transport. SSE (Server-Sent Events) is marked as **legacy** and should NOT be used for new deployments.

Impact on this manual:

- All nginx configurations in Chapter 2 are already optimized for Streamable HTTP (chunked transfer, no buffering)
- If you see references to "SSE" in older FastMCP tutorials, ignore them

- The `proxy_read_timeout` of 600s is for long-running tool execution, not for keeping an SSE connection alive

Key difference:

- SSE: Long-lived connection, server pushes events → requires high `proxy_read_timeout`, connection management
 - Streamable HTTP: Standard request/response with chunked encoding → simpler proxy config, no connection state issues
-

Correction 3: OAuth 2.1 is Mandatory for Remote MCP Servers

The MCP specification (2025-11-05 revision, updated 2025-06-18) **mandates OAuth 2.1** for all remote MCP servers. This includes:

1. **Dynamic Client Registration (DCR)** — MCP clients **MUST** be able to self-register
2. **PKCE** (S256 only) — Required on all authorization code flows
3. **Protected Resource Metadata** (RFC 9728) — For server discovery
4. **Resource Indicators** (RFC 8707) — For multi-tenant deployments

Authentication hierarchy for this manual:

- **Primary (Chapter 3, Pattern 3):** Full OAuth 2.1 + DCR — required for Claude, Cursor, and all public MCP clients
- **Secondary (Chapter 3, Pattern 2):** JWT validation — for internal service-to-service calls where you control both ends
- **Fallback (Chapter 3, Pattern 1):** Bearer tokens — development and trusted internal networks only

Anthropic Claude requires a hosted MCP endpoint implementing OAuth 2.1 with DCR before it can connect as a Custom Connector.

Correction 4: CVE-2026-48710 — Accurate Version Details

Field	Value
CVE ID	CVE-2026-48710
Disclosed	May 26, 2026
NVD Analysis	June 2, 2026
CVSS v3.1	6.5 (Medium) — AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
Affected	Starlette $\geq 0.8.3$, $< 1.0.1$
Also affects	FastAPI $< 0.116.0$ (uses Starlette internally)
Fixed in	Starlette $\geq 1.0.1$, FastAPI $\geq 0.116.0$
Name	"BadHost" — Host header auth bypass

Attack mechanism: A malformed Host header containing /, ?, or # characters causes `request.url.path` to differ from the actual routed path. Auth middleware that checks `request.url.path` (e.g., to skip auth for `/health`) can be bypassed.

Critical rule: Always use `request.scope["path"]` for security decisions, never `request.url.path`.

Correction 5: Prefect Horizon — Zero-Config Deployment Alternative

Before building your own nginx + Docker + CI/CD infrastructure, consider **Prefect Horizon** — the official managed platform from the FastMCP team.

What it provides:

- One-click deployment from a GitHub repo
- Built-in OAuth 2.1 with DCR (the spec-required auth, fully handled)
- TLS, autoscaling, per-request observability
- Free personal tier for individual developers

When to use Horizon:

- You want a production URL in under 5 minutes
- You don't want to manage nginx, TLS, or OAuth infrastructure

- You're building a public MCP server for Claude Custom Connectors

When to self-host (this manual's approach):

- You need full control over infrastructure
- You have compliance requirements (data residency, air-gapped environments)
- You're running internal-only MCP servers
- You want to avoid vendor lock-in

Quick start with Horizon:

```
# 1. Ensure your FastMCP server is in a GitHub repo
# 2. Visit https://horizon.prefect.io
# 3. Connect your repo
# 4. Every push to main auto-deploys

# Verify your server is Horizon-compatible:
fastmcp inspect server.py
```

Correction 6: Cloudflare Workers — Serverless MCP Deployment

For Chapter 6 (CI/CD), an alternative to VPS + Docker deployment is **Cloudflare Workers** — currently the most popular serverless path for MCP servers in the community.

Advantages:

- Global edge deployment (low latency worldwide)
- Built-in OAuth via Cloudflare Access
- No container management
- Pay-per-request pricing

Limitations:

- 30-second execution timeout (tools that run longer won't work)
- No persistent filesystem
- Limited to JavaScript/TypeScript (no Python FastMCP directly)
- Cold starts on first request

Best for: Lightweight MCP servers with fast tools (API wrappers, data lookup, text processing).

Not suitable for: Long-running tools (database migrations, file processing, ML inference) — use the Docker + VPS approach from this manual instead.

Example deployment:

```
# Using the Cloudflare MCP template
npx create-cloudflare@latest my-mcp-server --template=cloudflare/
ai/demos/remote-mcp-github-oauth

# Deploy
cd my-mcp-server
npx wrangler deploy
```

For Python-based FastMCP servers (this manual's focus), stick with the Docker + VPS pipeline in Chapter 6.

Correction 7: Additional OAuth DCR Errors for Chapter 8

These are the most commonly reported authentication issues in the MCP community (June 2026):

Error	Cause	Fix
registration_not_supported	Server doesn't implement /register endpoint	Implement DCR (see Chapter 3 Pattern 3) or use Horizon
invalid_client_metadata during DCR	redirect_uris format wrong (must be full URLs with scheme)	Use ["http://localhost:3000/callback"] not ["localhost:3000"]
unsupported_response_type: code	Server only supports implicit flow	Add "response_types_supported": ["code"] to OAuth metadata
Claude shows "Unable to connect"	Missing /.well-known/oauth-authorization-server endpoint	Implement OAuth metadata discovery (RFC 8414)
Token works in curl but not in Claude	Claude requires DCR flow, can't	Implement the full DCR + PKCE flow

Error	Cause	Fix
	use pre-configured client_id	

Correction 8: FastMCP 2.x to 3.x Migration Pitfalls

If upgrading from FastMCP 2.14.x to 3.x, watch for these breaking changes:

Area	2.x Behavior	3.x Change
Transport	SSE default	Streamable HTTP default
Auth	Manual middleware	Built-in auth= parameter
Session storage	In-memory only	Pluggable (Redis, etc.) via stateless mode
Tool naming	get_* prefix convention	list_* prefix (breaking rename)
CLI	fastmcp run	fastmcp run (same, but new flags)
Workers	Single-worker only for sessions	Multi-worker with --stateless flag
Dependencies	mcp SDK bundled	Separate fastmcp package

Migration command:

```
# Check for breaking changes
pip install fastmcp --upgrade --dry-run

# Test compatibility
fastmcp inspect server.py
```